

INTERNATIONAL STANDARD

NORME INTERNATIONALE



**OPC Unified Architecture –
Part 8: Data Access**

**Architecture unifiée OPC –
Partie 8: Accès aux données**



THIS PUBLICATION IS COPYRIGHT PROTECTED

Copyright © 2015 IEC, Geneva, Switzerland

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized in any form or by any means, electronic or mechanical, including photocopying and microfilm, without permission in writing from either IEC or IEC's member National Committee in the country of the requester. If you have any questions about IEC copyright or have an enquiry about obtaining additional rights to this publication, please contact the address below or your local IEC member National Committee for further information.

Droits de reproduction réservés. Sauf indication contraire, aucune partie de cette publication ne peut être reproduite ni utilisée sous quelque forme que ce soit et par aucun procédé, électronique ou mécanique, y compris la photocopie et les microfilms, sans l'accord écrit de l'IEC ou du Comité national de l'IEC du pays du demandeur. Si vous avez des questions sur le copyright de l'IEC ou si vous désirez obtenir des droits supplémentaires sur cette publication, utilisez les coordonnées ci-après ou contactez le Comité national de l'IEC de votre pays de résidence.

IEC Central Office
3, rue de Varembe
CH-1211 Geneva 20
Switzerland

Tel.: +41 22 919 02 11
Fax: +41 22 919 03 00
info@iec.ch
www.iec.ch

About the IEC

The International Electrotechnical Commission (IEC) is the leading global organization that prepares and publishes International Standards for all electrical, electronic and related technologies.

About IEC publications

The technical content of IEC publications is kept under constant review by the IEC. Please make sure that you have the latest edition, a corrigenda or an amendment might have been published.

IEC Catalogue - webstore.iec.ch/catalogue

The stand-alone application for consulting the entire bibliographical information on IEC International Standards, Technical Specifications, Technical Reports and other documents. Available for PC, Mac OS, Android Tablets and iPad.

IEC publications search - www.iec.ch/searchpub

The advanced search enables to find IEC publications by a variety of criteria (reference number, text, technical committee,...). It also gives information on projects, replaced and withdrawn publications.

IEC Just Published - webstore.iec.ch/justpublished

Stay up to date on all new IEC publications. Just Published details all new publications released. Available online and also once a month by email.

Electropedia - www.electropedia.org

The world's leading online dictionary of electronic and electrical terms containing more than 30 000 terms and definitions in English and French, with equivalent terms in 15 additional languages. Also known as the International Electrotechnical Vocabulary (IEV) online.

IEC Glossary - std.iec.ch/glossary

More than 60 000 electrotechnical terminology entries in English and French extracted from the Terms and Definitions clause of IEC publications issued since 2002. Some entries have been collected from earlier publications of IEC TC 37, 77, 86 and CISPR.

IEC Customer Service Centre - webstore.iec.ch/csc

If you wish to give us your feedback on this publication or need further assistance, please contact the Customer Service Centre: csc@iec.ch.

A propos de l'IEC

La Commission Electrotechnique Internationale (IEC) est la première organisation mondiale qui élabore et publie des Normes internationales pour tout ce qui a trait à l'électricité, à l'électronique et aux technologies apparentées.

A propos des publications IEC

Le contenu technique des publications IEC est constamment revu. Veuillez vous assurer que vous possédez l'édition la plus récente, un corrigendum ou amendement peut avoir été publié.

Catalogue IEC - webstore.iec.ch/catalogue

Application autonome pour consulter tous les renseignements bibliographiques sur les Normes internationales, Spécifications techniques, Rapports techniques et autres documents de l'IEC. Disponible pour PC, Mac OS, tablettes Android et iPad.

Recherche de publications IEC - www.iec.ch/searchpub

La recherche avancée permet de trouver des publications IEC en utilisant différents critères (numéro de référence, texte, comité d'études,...). Elle donne aussi des informations sur les projets et les publications remplacées ou retirées.

IEC Just Published - webstore.iec.ch/justpublished

Restez informé sur les nouvelles publications IEC. Just Published détaille les nouvelles publications parues. Disponible en ligne et aussi une fois par mois par email.

Electropedia - www.electropedia.org

Le premier dictionnaire en ligne de termes électroniques et électriques. Il contient plus de 30 000 termes et définitions en anglais et en français, ainsi que les termes équivalents dans 15 langues additionnelles. Egalement appelé Vocabulaire Electrotechnique International (IEV) en ligne.

Glossaire IEC - std.iec.ch/glossary

Plus de 60 000 entrées terminologiques électrotechniques, en anglais et en français, extraites des articles Termes et Définitions des publications IEC parues depuis 2002. Plus certaines entrées antérieures extraites des publications des CE 37, 77, 86 et CISPR de l'IEC.

Service Clients - webstore.iec.ch/csc

Si vous désirez nous donner des commentaires sur cette publication ou si vous avez des questions contactez-nous: csc@iec.ch.

INTERNATIONAL STANDARD

NORME INTERNATIONALE



**OPC Unified Architecture –
Part 8: Data Access**

**Architecture unifiée OPC –
Partie 8: Accès aux données**

INTERNATIONAL
ELECTROTECHNICAL
COMMISSION

COMMISSION
ELECTROTECHNIQUE
INTERNATIONALE

ICS 25.040.40; 35.100

ISBN 978-2-8322-2273-7

Warning! Make sure that you obtained this publication from an authorized distributor. Attention! Veuillez vous assurer que vous avez obtenu cette publication via un distributeur agréé.
--

CONTENTS

FOREWORD.....	4
1 Scope.....	6
2 Normative references.....	6
3 Terms, definitions and abbreviations	6
3.1 Terms and definitions	6
3.2 Abbreviations and symbols	7
4 Concepts.....	7
5 Model.....	8
5.1 General.....	8
5.2 SemanticsChanged	9
5.3 Variable Types	9
5.3.1 DataItemType	9
5.3.2 AnalogItemType	10
5.3.3 DiscreteItemType	11
5.3.4 ArrayItemType	13
5.4 Address Space model	18
5.5 Attributes of DataItems	19
5.6 DataTypes	20
5.6.1 Overview.....	20
5.6.2 Range.....	20
5.6.3 EUInformation	20
5.6.4 ComplexNumberType	21
5.6.5 DoubleComplexNumberType	22
5.6.6 AxisInformation	22
5.6.7 AxisScaleEnumeration.....	23
5.6.8 XVType.....	23
6 Data Access specific usage of Services	23
6.1 General.....	23
6.2 PercentDeadband	24
6.3 Data Access status codes	24
6.3.1 Overview.....	24
6.3.2 Operation level result codes	24
6.3.3 LimitBits.....	26
Figure 1 – OPC <i>DataItems</i> are linked to automation data	8
Figure 2 – <i>DataItem VariableType</i> hierarchy	9
Figure 3 – Graphical view of a <i>YArrayItem</i>	15
Figure 4 – Representation of DataItems in the AddressSpace	19
Table 1 – DataItemType definition	9
Table 2 – <i>AnalogItemType</i> definition	10
Table 3 – DiscreteItemType definition	11
Table 4 – TwoStateDiscreteType definition	12
Table 5 – MultiStateDiscreteType definition	12
Table 6 – MultiStateValueDiscreteType definition	13

Table 7 – ArrayItemType definition	14
Table 8 – YArrayItemType definition	14
Table 9 – YArrayItem item description	15
Table 10 – XYArrayItemType definition	16
Table 11 – ImageItemType definition	17
Table 12 – CubeltemType definition	17
Table 13 – NDimensionArrayItemType definition	18
Table 14 – <i>Range</i> DataType structure	20
Table 15 – <i>Range</i> definition	20
Table 16 – <i>EUInformation</i> DataType structure	20
Table 17 – <i>EUInformation</i> definition	20
Table 18 – Examples from the UNECE Recommendation	21
Table 19 – ComplexNumberType DataType structure	22
Table 20 – ComplexNumberType definition	22
Table 21 – DoubleComplexNumberType DataType structure	22
Table 22 – DoubleComplexNumberType definition	22
Table 23 – AxisInformation DataType structure	22
Table 24 – AxisScaleEnumeration values	23
Table 25 – AxisScaleEnumeration definition	23
Table 26 – XVType DataType structure	23
Table 27 – XVType definition	23
Table 28 – Operation level result codes for BAD data quality	25
Table 29 – Operation level result codes for UNCERTAIN data quality	25
Table 30 – Operation level result codes for GOOD data quality	25

INTERNATIONAL ELECTROTECHNICAL COMMISSION

OPC UNIFIED ARCHITECTURE –

Part 8: Data Access

FOREWORD

- 1) The International Electrotechnical Commission (IEC) is a worldwide organization for standardization comprising all national electrotechnical committees (IEC National Committees). The object of IEC is to promote international co-operation on all questions concerning standardization in the electrical and electronic fields. To this end and in addition to other activities, IEC publishes International Standards, Technical Specifications, Technical Reports, Publicly Available Specifications (PAS) and Guides (hereafter referred to as “IEC Publication(s)”). Their preparation is entrusted to technical committees; any IEC National Committee interested in the subject dealt with may participate in this preparatory work. International, governmental and non-governmental organizations liaising with the IEC also participate in this preparation. IEC collaborates closely with the International Organization for Standardization (ISO) in accordance with conditions determined by agreement between the two organizations.
- 2) The formal decisions or agreements of IEC on technical matters express, as nearly as possible, an international consensus of opinion on the relevant subjects since each technical committee has representation from all interested IEC National Committees.
- 3) IEC Publications have the form of recommendations for international use and are accepted by IEC National Committees in that sense. While all reasonable efforts are made to ensure that the technical content of IEC Publications is accurate, IEC cannot be held responsible for the way in which they are used or for any misinterpretation by any end user.
- 4) In order to promote international uniformity, IEC National Committees undertake to apply IEC Publications transparently to the maximum extent possible in their national and regional publications. Any divergence between any IEC Publication and the corresponding national or regional publication shall be clearly indicated in the latter.
- 5) IEC itself does not provide any attestation of conformity. Independent certification bodies provide conformity assessment services and, in some areas, access to IEC marks of conformity. IEC is not responsible for any services carried out by independent certification bodies.
- 6) All users should ensure that they have the latest edition of this publication.
- 7) No liability shall attach to IEC or its directors, employees, servants or agents including individual experts and members of its technical committees and IEC National Committees for any personal injury, property damage or other damage of any nature whatsoever, whether direct or indirect, or for costs (including legal fees) and expenses arising out of the publication, use of, or reliance upon, this IEC Publication or any other IEC Publications.
- 8) Attention is drawn to the Normative references cited in this publication. Use of the referenced publications is indispensable for the correct application of this publication.
- 9) Attention is drawn to the possibility that some of the elements of this IEC Publication may be the subject of patent rights. IEC shall not be held responsible for identifying any or all such patent rights.

International Standard IEC 62541-8 has been prepared by subcommittee 65E: Devices and integration in enterprise systems, of IEC technical committee 65: Industrial-process measurement, control and automation.

This second edition cancels and replaces the first edition published in 2011. This edition constitutes a technical revision.

This edition includes the following significant technical changes with respect to the previous edition:

- a) Clarified that deadband has to be between 0.0 and 100.0. Violations result in error `Bad_DeadbandFilterInvalid` (6.2)
- b) Added `VariableTypes` handling `ArrayItems` and `DataTypes` supporting this, including complex number types. These data types are required for complex analyzer devices but seem useful for other domains as well.

The text of this standard is based on the following documents:

CDV	Report on voting
65E/381/CDV	65E/407/RVD

Full information on the voting for the approval of this standard can be found in the report on voting indicated in the above table.

This publication has been drafted in accordance with the ISO/IEC Directives, Part 2.

A list of all parts of the IEC 62541 series, published under the general title *OPC Unified Architecture*, can be found on the IEC website.

The committee has decided that the contents of this publication will remain unchanged until the stability date indicated on the IEC web site under "<http://webstore.iec.ch>" in the data related to the specific publication. At this date, the publication will be

- reconfirmed,
- withdrawn,
- replaced by a revised edition, or
- amended.

IMPORTANT – The 'colour inside' logo on the cover page of this publication indicates that it contains colours which are considered to be useful for the correct understanding of its contents. Users should therefore print this document using a colour printer.

OPC UNIFIED ARCHITECTURE –

Part 8: Data Access

1 Scope

This part of IEC 62451 is part of the overall OPC Unified Architecture (OPC UA) standard series and defines the information model associated with Data Access (DA). It particularly includes additional *VariableTypes* and complementary descriptions of the *NodeClasses* and *Attributes* needed for Data Access, additional *Properties*, and other information and behaviour.

The complete address space model, including all *NodeClasses* and *Attributes* is specified in IEC 62541-3. The services to detect and access data are specified in IEC 62541-4.

2 Normative references

The following documents, in whole or in part, are normatively referenced in this document and are indispensable for its application. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

IEC TR 62541-1, *OPC Unified Architecture - Part 1: Overview and Concepts*

IEC 62541-3, *OPC unified architecture - Part 3: Address Space Model*

IEC 62541-4, *OPC unified architecture - Part 4: Services*

IEC 62541-5, *OPC unified architecture - Part 5: Information Model*

UN/CEFACT: **UNECE Recommendation N° 20**, *Codes for Units of Measure Used in International Trade*, available at http://www.unece.org/cefact/recommendations/rec_index.htm

3 Terms, definitions and abbreviations

3.1 Terms and definitions

For the purposes of this document, the terms and definitions given in IEC TR 62541-1, IEC 62541-3, and IEC 62541-4 as well as the following apply.

3.1.1

Dataltem

link to arbitrary, live automation data, that is, data that represents currently valid information

Note 1 to entry: Examples of such data are

- device data (such as temperature sensors),
- calculated data,
- status information (open/closed, moving),
- dynamically-changing system data (such as stock quotes),
- diagnostic data.

3.1.2

AnalogItem

DatalItems that represent continuously-variable physical quantities (e.g., length, temperature), in contrast to the digital representation of data in discrete items

Note 1 to entry: Typical examples are the values provided by temperature sensors or pressure sensors. OPC UA defines a specific *VariableType* to identify an *AnalogItem*. *Properties* describe the possible ranges of *AnalogItems*.

3.1.3

DiscreteItem

DatalItems that represent data that may take on only a certain number of possible values (e.g., OPENING, OPEN, CLOSING, CLOSED)

Note 1 to entry: Specific *VariableTypes* are used to identify *DiscreteItems* with two states or with multiple states. *Properties* specify the string values for these states.

3.1.4

ArrayItem

DatalItems that represent continuously-variable physical quantities and where each individual data point consists of multiple values represented by an array (e.g., the spectral response of a digital filter)

Note 1 to entry: Typical examples are the data provided by analyser devices. Specific *VariableTypes* are used to identify *ArrayItem* variants.

3.1.5

EngineeringUnits

units of measurement for *AnalogItems* that represent continuously-variable physical quantities (e.g., length, mass, time, temperature)

Note 1 to entry: This standard defines *Properties* to inform about the unit used for the *DatalItem* value and about the highest and lowest value likely to be obtained in normal operation.

3.2 Abbreviations and symbols

DA	Data Access
EU	Engineering Unit
UA	Unified Architecture

4 Concepts

Data Access deals with the representation and use of automation data in Servers.

Automation data can be located inside the *Server* or on I/O cards directly connected to the *Server*. It can also be located in sub-servers or on other devices such as controllers and input/output modules, connected by serial links via field buses or other communication links. OPC UA Data Access *Servers* provide one or more OPC UA Data Access *Clients* with transparent access to their automation data.

The links to automation data instances are called *DatalItems*. Which categories of automation data are provided is completely vendor-specific. Figure 1 illustrates how the *AddressSpace* of a *Server* might consist of a broad range of different *DatalItems*.

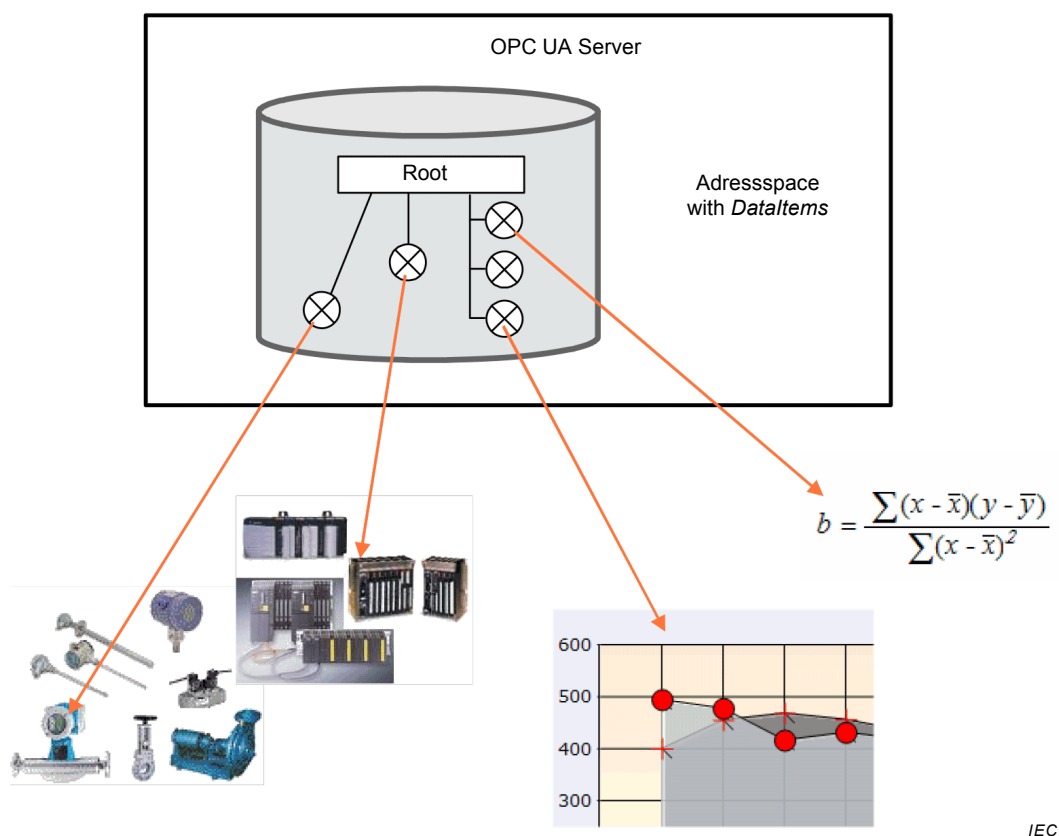


Figure 1 – OPC *Dataltems* are linked to automation data

Clients may read or write *Dataltems*, or monitor them for value changes. The *Services* needed for these operations are specified in IEC 62541-4. Changes are defined as a change in status (quality) or a change in value that exceeds a client-defined range called a *Deadband*. To detect the value change, the difference between the current value and the last reported value is compared to the *Deadband*.

5 Model

5.1 General

The *DataAccess* model extends the variable model by defining *VariableTypes*. The *DataltemType* is the base type. *ArrayItemtype*, *AnalogItemtype* and *DiscreteltemType* (and its *TwoState* and *MultiState* subtypes) are specializations. See Figure 2. Each of these *VariableTypes* can be further extended to form domain or server specific *Dataltems*.

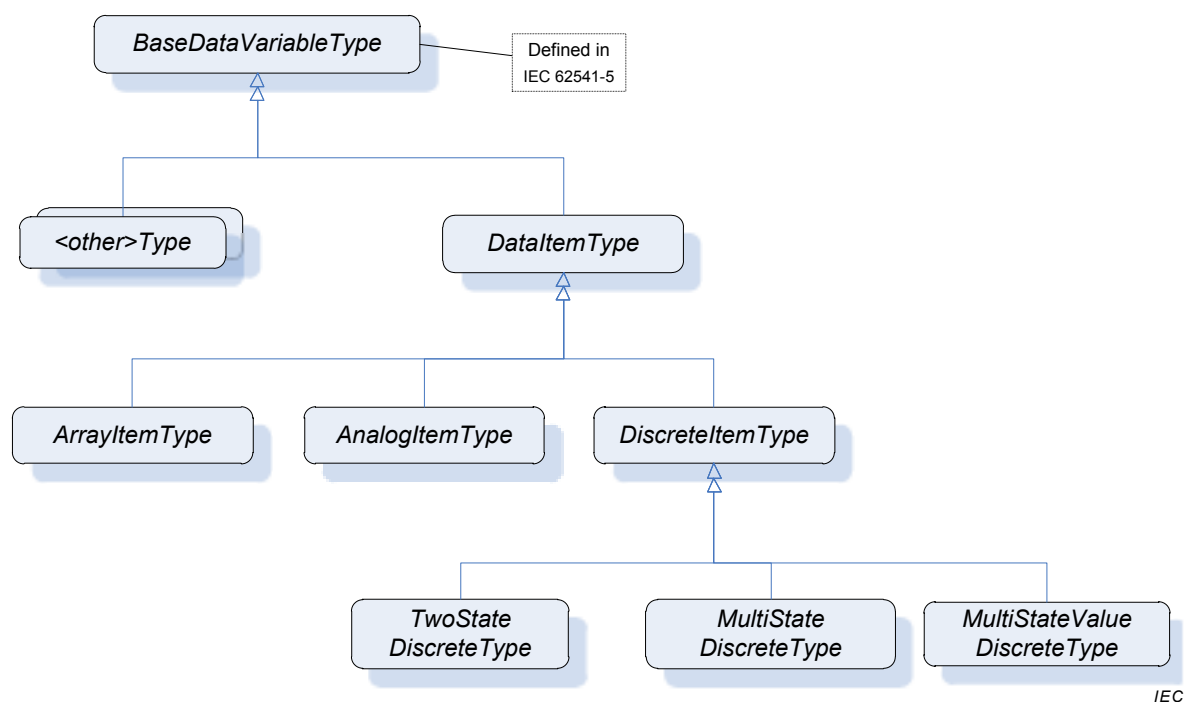


Figure 2 – DataItem VariableType hierarchy

5.2 SemanticsChanged

The *StatusCode* also contains an informational bit called *SemanticsChanged*.

Servers that implement Data Access shall set this Bit in notifications if certain *Properties* defined in this standard change. The corresponding *Properties* are specified individually for each *VariableType*.

Clients that use any of these *Properties* should re-read them before they process the data value.

5.3 Variable Types

5.3.1 DataItem Type

This *VariableType* defines the general characteristics of a *DataItem*. All other *DataItem* Types derive from it. The *DataItem Type* derives from the *BaseDataVariableType* and therefore shares the variable model as described in IEC 62541-3 and IEC 62541-5. It is formally defined in Table 1.

Table 1 – DataItem Type definition

Attribute	Value				
BrowseName	DataItem Type				
IsAbstract	False				
ValueRank	-2 (-2 = 'Any')				
Data Type	BaseDataVariableType				
References	NodeClass	BrowseName	Data Type	TypeDefinition	ModellingRule
Subtype of the <i>BaseDataVariableType</i> defined in IEC 62541-5; i.e the <i>Properties</i> of that type are inherited.					
HasSubtype	VariableType	AnalogItemType	Defined in 5.3.2		
HasSubtype	VariableType	DiscreteItemType	Defined in 5.3.3		
HasSubtype	VariableType	ArrayItemType	Defined in 5.3.4		
HasProperty	Variable	Definition	String	PropertyType	Optional
HasProperty	Variable	ValuePrecision	Double	PropertyType	Optional

Definition is a vendor-specific, human readable string that specifies how the value of this *DataItem* is calculated. *Definition* is non-localized and will often contain an equation that can be parsed by certain clients.

Example: *Definition*::= "(TempA - 25) + TempB"

ValuePrecision specifies the maximum precision that the *Server* can maintain for the item based on restrictions in the target environment.

ValuePrecision can be used for the following *DataTypes*:

- For Float and Double values it specifies the number of digits after the decimal place.
- For DateTime values it indicates the minimum time difference in nanoseconds. For example, a *ValuePrecision* of 20 000 000 defines a precision of 20 ms.

The *ValuePrecision Property* is an approximation that is intended to provide guidance to a *Client*. A *Server* is expected to silently round any value with more precision that it supports. This implies that a *Client* may encounter cases where the value read back from a *Server* differs from the value that it wrote to the *Server*. This difference shall be no more than the difference suggested by this *Property*.

5.3.2 AnalogItem Type

This *VariableType* defines the general characteristics of an *AnalogItem*. All other *AnalogItem* Types derive from it. The *AnalogItem Type* derives from the *DataItem Type*. It is formally defined in Table 2.

Table 2 – AnalogItem Type definition

Attribute	Value				
BrowseName	AnalogItem Type				
IsAbstract	False				
ValueRank	-2 (-2 = 'Any')				
DataType	Number				
References	NodeClass	BrowseName	DataType	TypeDefinition	ModellingRule
Subtype of the <i>DataItem Type</i> defined in 5.3.1 i.e the <i>Properties</i> of that type are inherited.					
HasProperty	Variable	InstrumentRange	Range	PropertyType	Optional
HasProperty	Variable	EURange	Range	PropertyType	Mandatory
HasProperty	Variable	EngineeringUnits	EUInformation	PropertyType	Optional

The following paragraphs describe the *Properties* of this *VariableType*. If the analog item's *Value* contains an array, the *Properties* shall apply to all elements in the array.

InstrumentRange defines the value range that can be returned by the instrument.

Example: *InstrumentRange*::= {-9999.9, 9999.9}

Although defined as optional, it is strongly recommended for *Servers* to support this *Property*. Without an *InstrumentRange* being provided, *Clients* will commonly assume the full range according to the *DataType*.

The *Range Data Type* is specified in 5.6.2.

EURange defines the value range likely to be obtained in normal operation. It is intended for such use as automatically scaling a bar graph display.

Sensor or instrument failure or deactivation can result in a returned item value which is actually outside of this range. *Client* software must be prepared to deal with this possibility. Similarly a *Client* may attempt to write a value that is outside of this range back to the server.

The exact behaviour (accept, reject, clamp, etc.) in this case is *Server*-dependent. However, in general *Servers* shall be prepared to handle this.

Example: *EURange* := { -200.0, 1400.0 }

See also 6.2 for a special monitoring filter (*PercentDeadband*) which is based on the engineering unit range.

EngineeringUnits specifies the units for the *DataItem*'s value (e.g., DEGC, hertz, seconds). The *EUInformation* type is specified in 5.6.3.

Important note: Understanding the units of a measurement value is essential for a uniform system. In an open system in particular where servers from different cultures might be used, it is essential to know what the units of measurement are. Based on such knowledge, values can be converted if necessary before being used. Therefore, although defined as optional, support of the *EngineeringUnits Property* is strongly advised.

OPC UA recommends using the “**Codes for Units of Measurement**” (see UN/CEFACT: **UNECE Recommendation N° 20**). The mapping to the *EngineeringUnits Property* is specified in 5.6.3.

EXAMPLE OF UNIT MIX-UP: In 1999, the Mars Climate Orbiter crashed into the surface of Mars. The main reason was a discrepancy over the units used. The navigation software expected data in newton second; the company who built the orbiter provided data in pound-force seconds. Another, less expensive, disappointment occurs when people used to British pints order a pint in the USA, only to be served what they consider a short measure.

The *StatusCode SemanticsChanged* bit shall be set if any of the *EURange* (could change the behaviour of a *Subscription* if a *PercentDeadband* filter is used) or *EngineeringUnits* (could create problems if the client uses the value to perform calculations) *Properties* are changed (see section 5.2 for additional information).

5.3.3 DiscreteItemType

5.3.3.1 General

This *VariableType* is an abstract type. That is, no instances of this type can exist. However, it might be used in a filter when browsing or querying. The *DiscreteItemType* derives from the *DataItemType* and therefore shares all of its characteristics. It is formally defined in Table 3.

Table 3 – DiscreteItemType definition

Attribute	Value				
BrowseName	DiscreteItemType				
IsAbstract	True				
ValueRank	-2 (-2 = 'Any')				
DataType	BaseDataType				
References	NodeClass	BrowseName	DataType	TypeDefinition	ModellingRule
Subtype of the <i>DataItemType</i> defined in 5.2; i.e the <i>Properties</i> of that type are inherited.					
HasSubtype	VariableType	TwoStateDiscreteType	Defined in 5.3.3.2		
HasSubtype	VariableType	MultiStateDiscreteType	Defined in 5.3.3.3		
HasSubtype	VariableType	MultiStateValueDiscreteType	Defined in 5.3.3.4		

5.3.3.2 TwoStateDiscreteType

This *VariableType* defines the general characteristics of a *DiscreteItem* that can have two states. The *TwoStateDiscreteType* derives from the *DiscreteItemType*. It is formally defined in Table 4.

Table 4 – TwoStateDiscreteType definition

Attribute	Value				
BrowseName	TwoStateDiscreteType				
IsAbstract	False				
ValueRank	-2 (-2 = 'Any')				
DataType	Boolean				
References	NodeClass	BrowseName	DataType	TypeDefinition	ModellingRule
Subtype of the <i>DiscreteItem</i> Type defined in 5.3.3; i.e the <i>Properties</i> of that type are inherited.					
HasProperty	Variable	TrueState	LocalizedText	PropertyType	Mandatory
HasProperty	Variable	FalseState	LocalizedText	PropertyType	Mandatory

TrueState contains a string to be associated with this *DataItem* when it is TRUE. This is typically used for a contact when it is in the closed (non-zero) state.

for example: "RUN", "CLOSE", "ENABLE", "SAFE", etc.

FalseState contains a string to be associated with this *DataItem* when it is FALSE. This is typically used for a contact when it is in the open (zero) state.

for example: "STOP", "OPEN", "DISABLE", "UNSAFE", etc.

If the item contains an array, then the *Properties* will apply to all elements in the array.

The *StatusCode SemanticsChanged* bit shall be set if any of the *FalseState* or *TrueState* (changes can cause misinterpretation by users or (scripting) programs) *Properties* are changed (see section 5.2 for additional information).

5.3.3.3 MultiStateDiscreteType

This *VariableType* defines the general characteristics of a *DiscreteItem* that can have more than two states. The *MultiStateDiscreteType* derives from the *DiscreteItem* Type. It is formally defined in Table 5.

Table 5 – MultiStateDiscreteType definition

Attribute	Value				
BrowseName	MultiStateDiscreteType				
IsAbstract	False				
ValueRank	-2 (-2 = 'Any')				
DataType	UInteger				
References	NodeClass	BrowseName	DataType	TypeDefinition	ModellingRule
Subtype of the <i>DiscreteItem</i> Type defined in 5.3.3; i.e the <i>Properties</i> of that type are inherited.					
HasProperty	Variable	EnumStrings	LocalizedText[]	PropertyType	Mandatory

EnumStrings is a string lookup table corresponding to sequential numeric values (0, 1, 2, etc.)

Example:

"OPEN"
 "CLOSE"
 "IN TRANSIT" etc.

Here the string "OPEN" corresponds to 0, "CLOSE" to 1 and "IN TRANSIT" to 2.

Clients should be prepared to handle item values outside of the range of the list; and robust servers should be prepared to handle writes of illegal values.

If the item contains an array then this lookup table shall apply to all elements in the array.

NOTE The *EnumStrings* property is also used for Enumeration *DataTypes* (for the specification of this *DataType*, see IEC 62541-3).

The *StatusCode SemanticsChanged* bit shall be set if the *EnumStrings* (changes can cause misinterpretation by users or (scripting) programs) *Property* is changed (see section 5.2 for additional information).

5.3.3.4 MultiStateValueDiscreteType

This *VariableType* defines the general characteristics of a *DiscreteItem* that can have more than two states and where the state values (the enumeration) does not consist of consecutive numeric values (may have gaps) or where the enumeration is not zero-based. The *MultiStateValueDiscreteType* derives from the *DiscreteItemType*. It is formally defined in Table 6.

Table 6 – MultiStateValueDiscreteType definition

Attribute	Value				
BrowseName	MultiStateValueDiscreteType				
IsAbstract	False				
ValueRank	Scalar				
DataType	Number				
References	NodeClass	BrowseName	DataType	TypeDefinition	ModellingRule
Subtype of the <i>DiscreteItemType</i> defined in 5.3.3; i.e the <i>Properties</i> of that type are inherited.					
HasProperty	Variable	EnumValues	See IEC 62541-3		Mandatory
HasProperty	Variable	ValueAsText	See IEC 62541-3		Mandatory

EnumValues is an array of *EnumValueType*. Each entry of the array represents one enumeration value with its integer notation, a human-readable representation, and help information. This represents enumerations with integers that are not zero-based or have gaps (e.g. 1, 2, 4, 8, 16). See IEC 62541-3 for the definition of this type. *MultiStateValueDiscrete Variables* expose the current integer notation in their *Value Attribute*. *Clients* will often read the *EnumValues Property* in advance and cache it to lookup a name or help whenever they receive the numeric representation.

MultiStateValueDiscrete Variables can have any numeric *Data Type*; this includes signed and unsigned integers from 8 to 64 Bit length.

The numeric representation of the current enumeration value is provided via the *Value Attribute* of the *MultiStateValueDiscrete Variable*. The *ValueAsText Property* provides the localized text representation of the enumeration value. It can be used by *Clients* only interested in displaying the text to subscribe to the *Property* instead of the *Value Attribute*.

5.3.4 ArrayItemType

5.3.4.1 General

This abstract *VariableType* defines the general characteristics of an *ArrayItem*. Values are exposed in an array but the content of the array represents a single entity like an image. Other *DataItems* might contain arrays that represent for example several values of several temperature sensors of a boiler.

ArrayItemType or its subtype shall only be used when the *Title* and *AxisScaleType Properties* can be filled with reasonable values. If this is not the case *DataItemType* and subtypes like *AnalogItemType*, which also support arrays, shall be used. The *ArrayItemType* is formally defined in Table 7.

Table 7 – ArrayItemType definition

Attribute	Value				
BrowseName	ArrayItemType				
IsAbstract	True				
ValueRank	0 (0 = OneOrMoreDimensions)				
DataType	BaseDataType				
References	NodeClass	BrowseName	DataType	TypeDefinition	ModellingRule
Subtype of the <i>DataItem</i> Type defined in 5.3.1; i.e the <i>Properties</i> of that type are inherited.					
HasSubtype	VariableType	YArrayItemType	Defined in 5.3.4.2		
HasSubtype	VariableType	XYArrayItemType	Defined in 5.3.4.3		
HasSubtype	VariableType	ImageItemType	Defined in 5.3.4.4		
HasSubtype	VariableType	CubeItemType	Defined in 5.3.4.5		
HasSubtype	VariableType	NDimensionArrayItemType	Defined in 5.3.4.6		
HasProperty	Variable	InstrumentRange	Range	PropertyType	Optional
HasProperty	Variable	EURange	Range	PropertyType	Mandatory
HasProperty	Variable	EngineeringUnits	EUInformation	PropertyType	Mandatory
HasProperty	Variable	Title	LocalizedText	PropertyType	Mandatory
HasProperty	Variable	AxisScaleType	AxisScaleEnumeration	PropertyType	Mandatory

InstrumentRange defines the range of the *Value* of the *ArrayItem*.

EURange defines the value range of the *ArrayItem* likely to be obtained in normal operation. It is intended for such use as automatically scaling a bar graph display.

EngineeringUnits holds the information about the engineering units of the *Value* of the *ArrayItem*.

For additional information about *InstrumentRange*, *EURange*, and *EngineeringUnits* see the description of *AnalogItem* Type in 5.3.2.

Title holds the user readable title of the *Value* of the *ArrayItem*.

AxisScaleType defines the scale to be used for the axis where the *Value* of the *ArrayItem* shall be displayed.

The *StatusCode SemanticsChanged* bit shall be set if any of the *InstrumentRange*, *EURange*, *EngineeringUnits* or *Title Properties* are changed (see 5.2 for additional information).

5.3.4.2 YArrayItemType

YArrayItemType represents a single-dimensional array of numerical values used to represent spectra or distributions where the x axis intervals are constant. *YArrayItemType* is formally defined in Table 8.

Table 8 – YArrayItemType definition

Attribute	Value				
BrowseName	YArrayItemType				
IsAbstract	False				
ValueRank	1				
DataType	BaseDataType				
ArrayDimensions	{0} (0 = UnknownSize)				
References	NodeClass	BrowseName	DataType	TypeDefinition	ModellingRule
Subtype of the <i>ArrayItem</i> Type defined in 5.3.4.1					
HasProperty	Variable	XAxisDefinition	AxisInformation	PropertyType	Mandatory

The *Value* of the *YArrayItem* contains the numerical values for the Y-Axis. *Engineering Units* and *Range* for the *Value* are defined by corresponding *Properties* inherited from the *ArrayItem* Type.

The *DataType* of this *VariableType* is restricted to *SByte*, *Int16*, *Int32*, *Int64*, *Float*, *Double*, *ComplexNumberType* and *DoubleComplexNumberType*.

The *XAxisDefinition* *Property* holds the information about the *Engineering Units* and *Range* for the X-Axis.

The *StatusCode* *SemanticsChanged* bit shall be set if any of the following five *Properties* are changed: *InstrumentRange*, *EURange*, *EngineeringUnits*, *Title* or *XAxisDefinition* (see 5.2 for additional information).

Figure 3 shows an example of how *Attributes* and *Properties* may be used in a graphical interface.

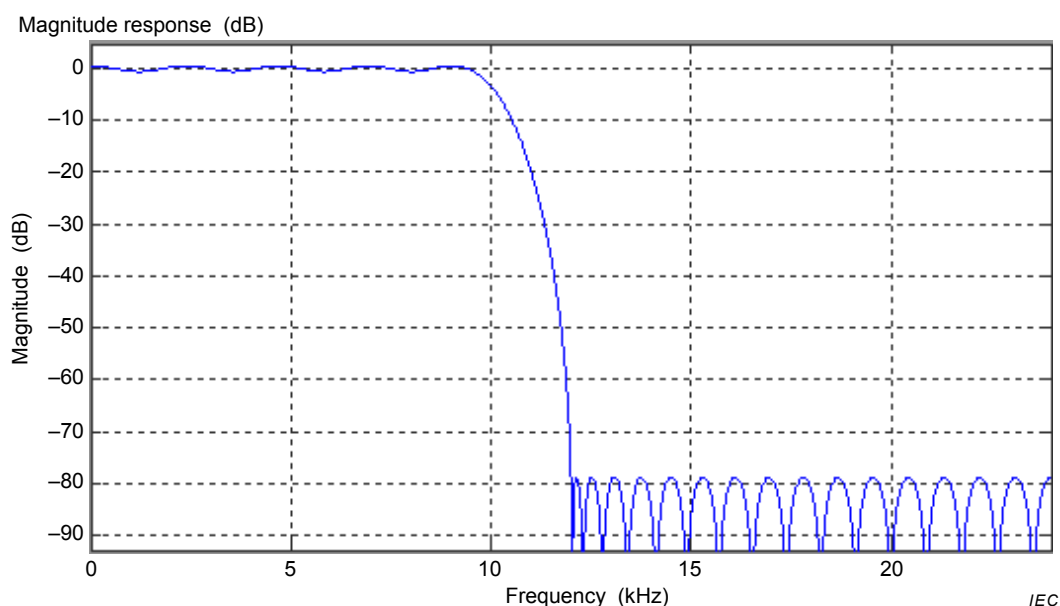


Figure 3 – Graphical view of a *YArrayItem*

Table 9 describes the values of each element presented in Figure 3.

Table 9 – *YArrayItem* item description

Attribute / Property	Item value
Description	Magnitude Response (dB)
axisScaleType	AxisScaleEnumeration.LINEAR 0
InstrumentRange.low	-90
InstrumentRange.high	5
EURange.low	-90
EURange.high	2
EngineeringUnits.namespaceUrl	http://www.opcfoundation.org/UA/units/un/cefact
EngineeringUnits.unitId	2N
EngineeringUnits.displayName	"en-us", "dB"
EngineeringUnits.description	"en-us", "decibel"
Title	Magnitude
XAxisDefinition.EngineeringUnits.namespaceUrl	http://www.opcfoundation.org/UA/units/un/cefact
XAxisDefinition.EngineeringUnits.unitId	kHz
XAxisDefinition.EngineeringUnits.displayName	"en-us", "kHz"
XAxisDefinition.EngineeringUnits.description	"en-us", "kilohertz"

Attribute / Property	Item value
XAxisDefinition.Range.low	0
XAxisDefinition.Range.high	25
XAxisDefinition.title	“en-us”, “Frequency”
XAxisDefinition.axisScaleType	AxisScaleEnumeration.LINEAR_0
XAxisDefinition.axisSteps	null

Interpretation notes:

- Not all elements of this table are used in the graphic.
- The X axis is displayed in reverse order, however, the *XAxisDefinition.Range.low* shall be lower than *XAxisDefinition.Range.high*. It is only a graphical representation that reverses the display order.
- There is a constant X axis

5.3.4.3 XYArrayType

XYArrayType represents a vector of *XVType* values like a list of peaks, where *XVType.x* is the position of the peak and *XVType.value* is its intensity. *XYArrayType* is formally defined in Table 10.

Table 10 – XYArrayType definition

Attribute	Value				
BrowseName	XYArrayType				
IsAbstract	False				
ValueRank	1				
DataType	XVType (defined in 5.6.8)				
References	NodeClass	BrowseName	DataType	TypeDefinition	ModellingRule
Subtype of the <i>ArrayType</i> defined in 5.3.4.1					
HasProperty	Variable	XAxisDefinition	AxisInformation	PropertyType	Mandatory

The *Value* of the *XYArrayType* contains an array of structures (*XVType*) where each structure specifies the position for the X-Axis (*XVType.x*) and the value itself (*XVType.value*), used for the Y-Axis. Engineering units and range for the *Value* are defined by corresponding *Properties* inherited from the *ArrayType*.

XAxisDefinition Property holds the information about the *Engineering Units* and *Range* for the X-Axis.

The *axisSteps* of *XAxisDefinition* shall be set to NULL because it is not used.

The *StatusCode SemanticsChanged* bit shall be set if any of the *InstrumentRange*, *EURange*, *EngineeringUnits*, *Title* or *XAxisDefinition Properties* are changed (see 5.2 for additional information).

5.3.4.4 ImageItemType

ImageItemType defines the general characteristics of an *ImageItem* which represents a matrix of values like an image, where the pixel position is given by X which is the column and Y the row. The value is the pixel intensity.

ImageItemType is formally defined in Table 11.

Table 11 – ImageItem type definition

Attribute	Value				
BrowseName	ImageItem type				
IsAbstract	False				
ValueRank	2 (2 = two dimensional array)				
DataType	BaseDataType				
References	NodeClass	BrowseName	DataType	TypeDefinition	ModellingRule
Subtype of the <i>ArrayItem</i> type defined in 5.3.4.1					
HasProperty	Variable	XAxisDefinition	AxisInformation	PropertyType	Mandatory
HasProperty	Variable	YAxisDefinition	AxisInformation	PropertyType	Mandatory

Engineering units and range for the *Value* are defined by corresponding *Properties* inherited from the *ArrayItem* type.

The *DataType* of this *VariableType* is restricted to SByte, Int16, Int32, Int64, Float, Double, ComplexNumberType and DoubleComplexNumberType.

The *ArrayDimensions* Attribute for *Variables* of this type or subtypes shall use the first entry in the array ([0]) to define the number of columns and the second entry ([1]) to define the number of rows, assuming the size of the matrix is not dynamic.

XAxisDefinition Property holds the information about the engineering units and range for the X-Axis.

YAxisDefinition Property holds the information about the engineering units and range for the Y-Axis.

The *StatusCode.SemanticsChanged* bit shall be set if any of the *InstrumentRange*, *EURange*, *EngineeringUnits*, *Title*, *XAxisDefinition* or *YAxisDefinition* Properties are changed.

5.3.4.5 CubelItem type

CubelItem type represents a cube of values like a spatial particle distribution, where the particle position is given by X which is the column, Y the row and Z the depth. In the example of a spatial partial distribution, the value is the particle size. *CubelItem* type is formally defined in Table 12.

Table 12 – CubelItem type definition

Attribute	Value				
BrowseName	CubelItem type				
IsAbstract	False				
ValueRank	3 (3 = three dimensional array)				
DataType	BaseDataType				
References	NodeClass	BrowseName	DataType	TypeDefinition	ModellingRule
Subtype of the <i>ArrayItem</i> type defined in 5.3.4.1					
HasProperty	Variable	XAxisDefinition	AxisInformation	PropertyType	Mandatory
HasProperty	Variable	YAxisDefinition	AxisInformation	PropertyType	Mandatory
HasProperty	Variable	ZAxisDefinition	AxisInformation	PropertyType	Mandatory

Engineering units and range for the *Value* are defined by corresponding *Properties* inherited from the *ArrayItem* type.

The *DataType* of this *VariableType* is restricted to SByte, Int16, Int32, Int64, Float, Double, ComplexNumberType and DoubleComplexNumberType.

The *ArrayDimensions Attribute* for *Variables* of this type or subtypes should use the first entry in the array ([0]) to define the number of columns, the second entry ([1]) to define the number of rows, and the third entry ([2]) define the number of steps in the Z axis, assuming the size of the matrix is not dynamic.

XAxisDefinition Property holds the information about the engineering units and range for the X-Axis.

YAxisDefinition Property holds the information about the engineering units and range for the Y-Axis.

ZAxisDefinition Property holds the information about the engineering units and range for the Z-Axis.

The *StatusCode SemanticsChanged* bit shall be set if any of the *InstrumentRange*, *EURange*, *EngineeringUnits*, *Title*, *XAxisDefinition*, *YAxisDefinition* or *ZAxisDefinition Properties* are changed (see 5.2 for additional information).

5.3.4.6 NDimensionArrayItemType

This *VariableType* defines a generic multi-dimensional *ArrayItem*.

This approach minimizes the number of types however it may be proved more difficult to utilize for control system interactions.

NDimensionArrayItemType is formally defined in Table 13.

Table 13 – NDimensionArrayItemType definition

Attribute	Value				
BrowseName	NdimensionArrayItemType				
IsAbstract	False				
ValueRank	0 (0 = OneOrMoreDimensions)				
DataType	BaseDataType				
References	NodeClass	BrowseName	DataType	TypeDefinition	ModellingRule
Subtype of the <i>ArrayItemType</i> defined in 5.3.4.1					
HasProperty	Variable	AxisDefinition	AxisInformation []	PropertyType	Mandatory

The *DataType* of this *VariableType* is restricted to SByte, Int16, Int32, Int64, Float, Double, ComplexNumberType and DoubleComplexNumberType.

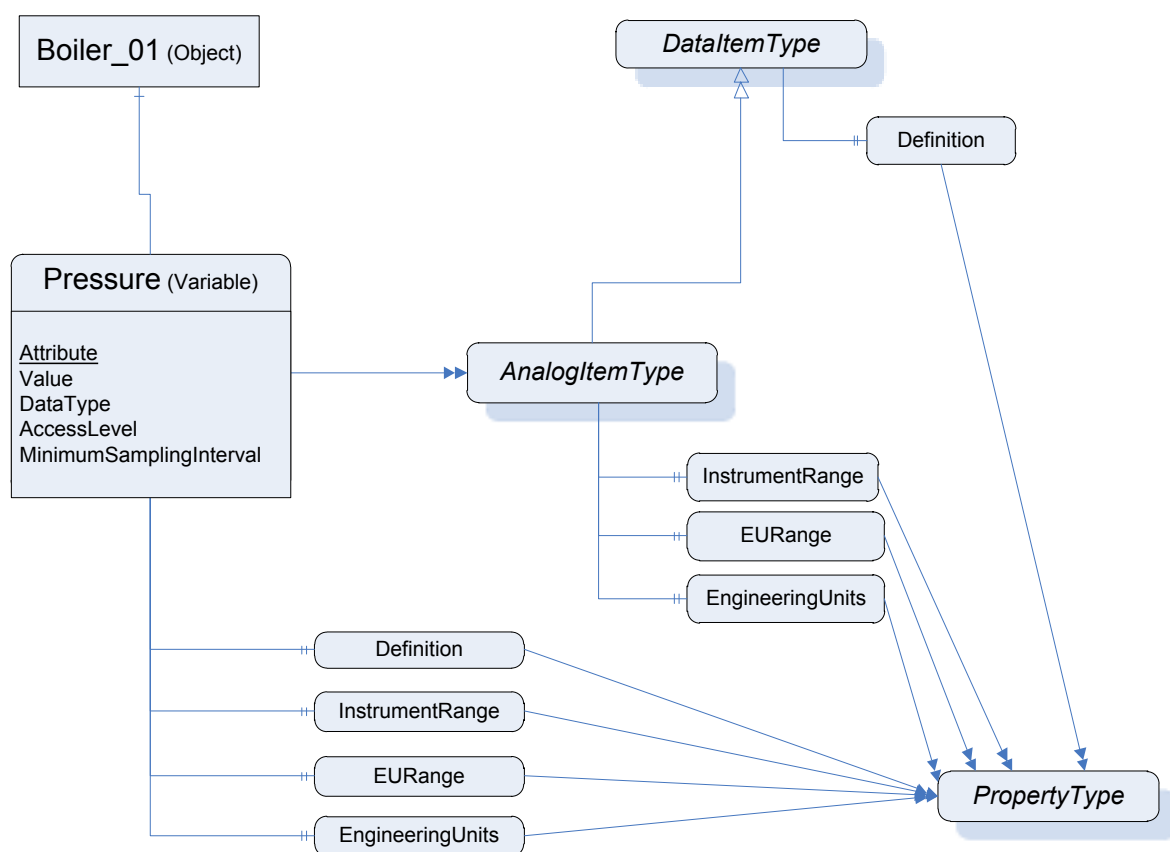
AxisDefinition Property holds the information about the *Engineering Units* and *Range* for all axis.

The *StatusCode SemanticsChanged* bit shall be set if any of the *InstrumentRange*, *EURange*, *EngineeringUnits*, *Title* or *AxisDefinition Properties* are changed (see 5.2 for additional information).

5.4 Address Space model

DataItems are always defined as data components of other *Nodes* in the *AddressSpace*. They are never defined by themselves. A simple example of a container for *DataItems* would be a “Folder Object” but it can be an *Object* of any other type.

Figure 4 illustrates the basic *AddressSpace* model of a *DataItem*, in this case an *AnalogItem*.



IEC

Figure 4 – Representation of Dataltems in the AddressSpace

Each *DataItem* is represented by a *DataVariable* with a specific set of *Attributes*. The *TypeDefinition* reference indicates the type of the *DataItem* (in this case the *AnalogItem Type*). Additional characteristics of *Dataltems* are defined using *Properties*. The *VariableTypes* in 5.2 specify which properties may exist. These *Properties* have been found to be useful for a wide range of Data Access clients. *Servers* that want to disclose similar information should use the OPC-defined *Property* rather than one that is vendor-specific.

The above figure shows only a subset of *Attributes* and *Properties*. Other *Attributes* that are defined for *Variables* in IEC 62541-3 (e.g., *Description*) may also be available.

5.5 Attributes of Dataltems

This subclause lists the *Attributes* of *Variables* that have particular importance for Data Access. They are specified in detail in IEC 62541-3. The following *Attributes* are particularly important for Data Access:

- Value
- DataType
- AccessLevel
- MinimumSamplingInterval

Value is the most recent value of the *Variable* that the *Server* has. Its data type is defined by the *DataType Attribute*. The *AccessLevel Attribute* defines the *Server's* basic ability to access current data and *MinimumSamplingInterval* defines how current the data is.

When a client requests the *Value Attribute* for reading or monitoring, the *Server* will always return a *StatusCode* (the quality and the *Server's* ability to access/provide the value) and,

optionally, a *ServerTimestamp* and/or a *SourceTimestamp* – based on the *Client's* request. See IEC 62541-4 for details on *StatusCodes* and the meaning of the two timestamps. Specific status codes for Data Access are defined in 6.3.

5.6 DataTypes

5.6.1 Overview

Following is a description of the *DataTypes* defined in this specification.

DataTypes like *String*, *Boolean*, *Double* or *LocalizedText* are defined in IEC 62541-3. Their representation is specified in IEC 62541-5.

5.6.2 Range

This structure defines the *Range* for a value. Its elements are defined in Table 14.

Table 14 – Range DataType structure

Name	Type	Description
Range	structure	
low	Double	Lowest value in the range.
high	Double	Highest value in the range.

Its representation in the *AddressSpace* is defined in Table 15

Table 15 – Range definition

Attributes	Value
BrowseName	Range

5.6.3 EUInformation

This structure contains information about the *EngineeringUnits*. Its elements are defined in Table 16.

Table 16 – EUInformation DataType structure

Name	Type	Description
EUInformation	structure	
namespaceUri	String	Identifies the organization (company, standards organization) that defines the <i>EUInformation</i> .
unitId	Int32	Identifier for programmatic evaluation. –1 is used if a <i>unitId</i> is not available.
displayName	LocalizedText	The <i>displayName</i> of the engineering unit is typically the abbreviation of the engineering unit, for example "h" for hour or "m/s" for meter per second.
description	LocalizedText	Contains the full name of the engineering unit such as "hour" or "meter per second".

Its representation in the *AddressSpace* is defined in Table 17

Table 17 – EUInformation definition

Attributes	Value
BrowseName	EUInformation

To facilitate interoperability, OPC UA specifies how to apply the widely accepted “**Codes for Units of Measurement**” published by the “United Nations Centre for Trade Facilitation and Electronic Business” (see UN/CEFACT: **UNECE Recommendation N° 20**). It uses and is based on the International System of Units (SI Units) but in addition provides a fixed code that

can be used for automated evaluation. This recommendation has been accepted by many industries on a global basis.

Table 18 contains a small excerpt of the published Annex with Code Lists:

Table 18 – Examples from the UNECE Recommendation

Excerpt from Recommendation N°. 20, Annex 1			
Common Code	Name	Conversion Factor	Symbol
C81	radian		rad
C25	milliradian	10^{-3} rad	mrاد
MMT	millimetre	10^{-3} m	mm
HMT	hectometre	10^2 m	hm
KTM	kilometre	10^3 m	km
KMQ	kilogram per cubic metre	kg/m ³	kg/m ³
FAH	degree Fahrenheit	$5/9 \times K$	°F
J23	degree Fahrenheit per hour	$1,543\,210 \times 10^{-4}$ K/s	°F/h

Specific columns of this table shall be used to create the *EUInformation* structure as defined by the following rules:

- The **Common Code** is represented as an alphanumeric variable length of 3 characters. It shall be used for the *EUInformation.unitId*. The following pseudo code specifies the algorithm to convert the Common Code into an Int32 as needed for *EUInformation.unitId*:

```
Int32 unitId = 0;
Int32 c;
for (i=0; i<=3;i++)
{
    c = CommonCode[i];
    if (c == 0) break;           // end of Common Code
    unitId = unitId << 8;
    unitId = unitId | c;
}
```

- The **Symbol** field shall be copied to the *EUInformation.displayName*. The *localeId* field of *EUInformation.displayName* shall be empty.
- The **Name** field shall be used for *EUInformation.description*. If the name is copied, then the *localeId* field of *EUInformation.description* shall be empty. If the name is localized then the *localeId* field shall specify the correct locale.

The *EUInformation.namespaceUri* shall be
<http://www.opcfoundation.org/UA/units/un/cefact>.

NOTE It will be advantageous to use Recommendation N°. 20 as specified, because it can be programmatically interpreted by generic OPC UA *Clients*. However, the *EUInformation* structure has been defined such that other standards bodies can incorporate their engineering unit definitions into OPC UA. If *Servers* use such an approach then they shall identify this standards body by using a proper *namespaceUri* in *EUInformation.namespaceUri*.

5.6.4 ComplexNumberType

This structure defines float IEEE 32 bits complex value. Its elements are defined in Table 19.

Table 19 – ComplexNumberType DataType structure

Name	Type	Description
ComplexNumberType	structure	
real	Float	Value real part
imaginary	Float	Value imaginary part

Its representation in the *AddressSpace* is defined in Table 20

Table 20 – ComplexNumberType definition

Attributes	Value
BrowseName	ComplexNumberType

5.6.5 DoubleComplexNumberType

This structure defines double IEEE 64 bits complex value. Its elements are defined in Table 21.

Table 21 – DoubleComplexNumberType DataType structure

Name	Type	Description
DoubleComplexNumberType	structure	
real	Double	Value real part
imaginary	Double	Value imaginary part

Its representation in the *AddressSpace* is defined in Table 22.

Table 22 – DoubleComplexNumberType definition

Attributes	Value
BrowseName	DoubleComplexNumberType

5.6.6 AxisInformation

This structure defines the information for auxiliary axis for *ArrayItemType Variables*.

There are three typical uses of this structure:

- The step between points is constant and can be predicted using the range information and the number of points. In this case, *axisSteps* can be set to NULL.
- The step between points is not constant, but remains the same for a long period of time (from acquisition to acquisition for example). In this case, *axisSteps* contains the value of each step on the axis.
- The step between points is not constant and changes at every update. In this case, a type like *XYArrayType* shall be used and *axisSteps* is set to NULL.

Its elements are defined in Table 23.

Table 23 – AxisInformation DataType structure

Name	Type	Description
AxisInformation	structure	
engineeringUnits	EUInformation	Holds the information about the engineering units for a given axis.
eURange	Range	Limits of the range of the axis
title	Localizedtext	User readable axis title, useful when the units are %, the Title may be "Particle size distribution"
axisScaleType	AxisScaleEnumeration	LINEAR, LOG, LN, defined by AxisSteps
axisSteps	Double[]	Specific value of each axis steps, may be set to "Null" if not used

When the steps in the axis are constant, *axisSteps* may be set to “Null” and in this case, the *Range* limits are used to compute the steps. The number of steps in the axis comes from the parent *ArrayItem.ArrayDimensions*.

5.6.7 AxisScaleEnumeration

This enumeration identifies on which type of axis the data shall be displayed. Its values are defined in Table 24.

Table 24 – AxisScaleEnumeration values

Value	Description
LINEAR_0	Linear scale
LOG_1	Log base 10 scale
LN_2	Log base e scale

Its representation in the *AddressSpace* is defined in Table 25.

Table 25 – AxisScaleEnumeration definition

Attributes	Value
BrowseName	AxisScaleEnumeration

5.6.8 XVType

This structure defines a physical value relative to a X axis and it is used as the *DataType* of the Value of *XYArrayItemType*. For details see 5.3.4.3.

Many devices can produce values that can perfectly be represented with a float IEEE 32 bits but, they can position them on the X axis with an accuracy that requires double IEEE 64 bits. For example, the peak value in an absorbance spectrum where the amplitude of the peak can be represented by a float IEEE 32 bits, but its frequency position required 10 digits which implies the use of a double IEEE 64 bits.

Its elements are defined in Table 26.

Table 26 – XVType DataType structure

Name	Type	Description
XVType	structure	
x	Double	Position on the X axis of this value
value	Float	The value itself

Its representation in the *AddressSpace* is defined in Table 27.

Table 27 – XVType definition

Attributes	Value
BrowseName	XVType

6 Data Access specific usage of Services

6.1 General

IEC 62541-4 specifies the complete set of services. The services needed for the purpose of *DataAccess* are:

- The *View* service set and *Query* service set to detect *DataItems*, and their *Properties*.
- The *Attribute* service set to read or write *Attributes* and in particular the value *Attribute*.

- The *MonitoredItem* and *Subscription* service set to set up monitoring of *DataItems* and to receive data change notifications.

6.2 PercentDeadband

The *DataChangeFilter* in IEC 62541-4 defines the conditions under which a data change notification shall be reported. This filter contains a *deadbandValue* which can be of type *AbsoluteDeadband* or *PercentDeadband*. IEC 62541-4 already specifies the behaviour of the *AbsoluteDeadband*. This sub-clause specifies the behaviour of the *PercentDeadband* type.

DeadbandType = PercentDeadband

For this type of deadband the *deadbandValue* is defined as the percentage of the *EURange*. That is, it applies only to *AnalogItems* with an *EURange Property* that defines the typical value range for the item. This range shall be multiplied with the *deadbandValue* and then compared to the actual value change to determine the need for a data change notification. The following pseudo code shows how the deadband is calculated:

```
DataChange if (absolute value of (last cached value - current value) >
               (deadbandValue/100.0) * ((high-low) of EURange))
```

The range of the *deadbandValue* is from 0.0 to 100.0 Percent. Specifying a *deadbandValue* outside of this range will be rejected and reported with the *StatusCode* *Bad_DeadbandFilterInvalid* (see Table 28).

If the Value of the *MonitoredItem* is an array, then the deadband calculation logic shall be applied to each element of the array. If an element that requires a *DataChange* is found, then no further deadband checking is necessary and the entire array shall be returned.

6.3 Data Access status codes

6.3.1 Overview

This subclause defines additional codes and rules that apply to the *StatusCode* when used for Data Access values.

The general structure of the *StatusCode* is specified in IEC 62541-4 and includes a set of common operational result codes that also apply to Data Access.

6.3.2 Operation level result codes

Certain conditions under which a *Variable* value was generated are only valid for automation data and in particular for device data; they are similar, but are slightly more generic than the description of data quality in the various fieldbus specifications.

In the following, Table 28 contains codes with BAD severity which indicates a failure.

Table 29 contains codes with UNCERTAIN severity which indicates that the value has been generated under sub-normal conditions.

Table 30 contains GOOD (success) codes.

Note again, that these are the codes that are specific for Data Access and supplement the codes that apply to all types of data which are defined in IEC 62541-4.

Table 28 – Operation level result codes for BAD data quality

Symbolic Id	Description
Remarks: The StatusCode Bad is defined in IEC 62541-4. It shall be used when there is no special reason why the Value is Bad.	
Bad_ConfigurationError	There is a problem with the configuration that affects the usefulness of the value.
Bad_NotConnected	The variable should receive its value from another variable, but has never been configured to do so.
Bad_DeviceFailure	There has been a failure in the device/data source that generates the value that has affected the value.
Bad_SensorFailure	There has been a failure in the sensor from which the value is derived by the device/data source. The limits bits are used to define if the limits of the value have been reached.
Remarks: Bad_NoCommunication is defined in IEC 62541-4. It shall be used when communications to the data source is defined, but not established, and there is no last known value available.	
Bad_OutOfService	The source of the data is not operational.
Bad_LastKnown	OPC UA requires that the <i>Server</i> shall return a Null value when the <i>Severity</i> is Bad. Therefore, the Fieldbus code "Bad_LastKnown" shall be mapped to Uncertain_NoCommunicationLastUsable.
Bad_DeadbandFilterInvalid	The specified <i>PercentDeadband</i> is not between 0.0 and 100.0 or a <i>PercentDeadband</i> is not supported, since an <i>EURange</i> is not configured.
Remarks: Bad_WaitingForInitialData is defined in IEC 62541-4.	

Table 29 – Operation level result codes for UNCERTAIN data quality

Symbolic Id	Description
Remarks: The StatusCode Uncertain is defined in IEC 62541-4. It shall be used when there is no special reason why the Value is Uncertain.	
Uncertain_NoCommunicationLastUsable	Communication to the data source has failed. The variable value is the last value that had a good quality and it is uncertain whether this value is still current. The server timestamp in this case is the last time that the communication status was checked. The time at which the value was last verified to be true is no longer available.
Uncertain_LastUsableValue	Whatever was updating this value has stopped doing so. This happens when an input variable is configured to receive its value from another variable and this configuration is cleared after one or more values have been received. This status/substatus is not used to indicate that a value is stale. Stale data can be detected by the client looking at the timestamps.
Uncertain_SubstituteValue	The value is an operational value that was manually overwritten.
Uncertain_InitialValue	The value is an initial value for a variable that normally receives its value from another variable. This status/substatus is set only during configuration while the variable is not operational (while it is out-of-service).
Uncertain_SensorNotAccurate	The value is at one of the sensor limits. The Limits bits define which limit has been reached. Also set if the device can determine that the sensor has reduced accuracy (e.g. degraded analyzer), in which case the Limits bits indicate that the value is not limited.
Uncertain_EngineeringUnitsExceeded	The value is outside of the range of values defined for this parameter. The Limits bits indicate which limit has been reached or exceeded.
Uncertain_SubNormal	The value is derived from multiple sources and has less than the required number of <u>Good</u> sources.

Table 30 – Operation level result codes for GOOD data quality

Symbolic Id	Description
Remarks: The StatusCode Good is defined in IEC 62541-4. It shall be used when there are no special conditions.	
Good_LocalOverride	The value has been Overridden. Typically this means the input has been disconnected and a manually-entered value has been "forced".

6.3.3 LimitBits

The bottom 16 bits of the *StatusCode* are bit flags that contain additional information, but do not affect the meaning of the *StatusCode*. Of particular interest for *DataItems* is the *LimitBits* field. In some cases, such as sensor failure it can provide useful diagnostic information.

Servers that do not support Limit have to set this field to 0.

SOMMAIRE

AVANT-PROPOS.....	30
1 Domaine d'application.....	32
2 Références normatives	32
3 Termes, définitions et abréviations	32
3.1 Termes et définitions	32
3.2 Abréviations et symboles.....	33
4 Concepts.....	33
5 Modèle	34
5.1 Généralités	34
5.2 Modifications Sémantiques (SemanticsChanged)	35
5.3 Types de Variables	35
5.3.1 Type d'Élément de Données (DataItemType)	35
5.3.2 Type d'Élément Analogique (AnalogItemType)	36
5.3.3 Type d'Élément Discret (DiscreteItemType)	38
5.3.4 Type d'élément de matrice (ArrayItemType).....	41
5.4 Modèle d'Espace d'adresses	47
5.5 Attributs des Éléments de Données	48
5.6 Types de Données	49
5.6.1 Aperçu général.....	49
5.6.2 Plage	49
5.6.3 Information EU	49
5.6.4 Type de Nombre Complexe (ComplexNumberType)	51
5.6.5 Type de Nombre Complexe Double (DoubleComplexNumberType).....	51
5.6.6 Informations sur l'Axe (AxisInformation).....	51
5.6.7 Énumération de Facteur d'Échelle selon un Axe (AxisScaleEnumeration)	52
5.6.8 Type XV (XVType)	52
6 Utilisation particulière des Services d'Accès aux Données	53
6.1 Généralités	53
6.2 Pourcentage de Bande morte (PercentDeadBand)	53
6.3 Codes de statuts d'Accès aux Données	54
6.3.1 Aperçu général.....	54
6.3.2 Codes de résultats de niveau opérationnel.....	54
6.3.3 Bits de limites (LimitBits)	56
Figure 1 – <i>Éléments de Données</i> OPC reliés aux données d'automatisation.....	34
Figure 2 – Hiérarchie du Type de Variable <i>Élément de Données</i>	35
Figure 3 – Représentation graphique d'un <i>Élément de Matrice Y</i>	43
Figure 4 – Représentation des <i>Éléments de Données</i> dans l' <i>Espace d'adresses</i>	48
Tableau 1 – Définition du Type d'Élément de Données	36
Tableau 2 – Définition du <i>Type d'Élément Analogique</i>	37
Tableau 3 – Définition du Type d'Élément Discret.....	38
Tableau 4– Définition du Type Discret à Deux États	39
Tableau 5 – Définition du Type Discret à États Multiples.....	39

Tableau 6 – Définition du Type Discret à Valeurs à États Multiples	40
Tableau 7 – Définition du <i>Type d'Élément de Matrice</i>	41
Tableau 8 – Définition du Type d'Élément de Matrice Y	42
Tableau 9 – Description de l' <i>Élément de Matrice Y</i>	43
Tableau 10 – Définition du Type d'Élément de Matrice XY	44
Tableau 11 – Définition du Type d'Élément d'Image.....	45
Tableau 12 – Définition du Type d'Élément de Cube	46
Tableau 13 – Définition du Type d'Élément de Matrice à N Dimensions	47
Tableau 14 – Structure du DataType (Type de Données) « <i>Plage</i> ».....	49
Tableau 15 – Définition de <i>Plage</i>	49
Tableau 16 – Structure du DataType (Type de Données) <i>Information EU</i>	49
Tableau 17 – Définition de l' <i>Information EU</i>	50
Tableau 18 – Exemples issus de la Recommandation UNECE	50
Tableau 19 – Structure du Type de Données Type de Nombre Complexe.....	51
Tableau 20 – Définition du Type de Nombre Complexe	51
Tableau 21 – Structure du Type de Données Type de Nombre Complexe Double	51
Tableau 22 – Définition du Type de Nombre Complexe Double	51
Tableau 23 – Structure du Type de Données Informations sur l'Axe	52
Tableau 24 – Valeurs de l'Énumération de Facteur d'Échelle selon un Axe	52
Tableau 25 – Définition de l'Énumération de Facteur d'Échelle selon un Axe	52
Tableau 26 – Structure du Type de Données Type XV	53
Tableau 27 – Définition du Type XV	53
Tableau 28 – Codes de résultats de niveau opérationnel MAUVAIS	54
Tableau 29 – Codes de résultats de niveau opérationnel DOUTEUX	55
Tableau 30 – Codes de résultats de niveau opérationnel BON	56

COMMISSION ÉLECTROTECHNIQUE INTERNATIONALE

ARCHITECTURE UNIFIÉE OPC –

Partie 8: Accès aux données

AVANT-PROPOS

- 1) La Commission Electrotechnique Internationale (IEC) est une organisation mondiale de normalisation composée de l'ensemble des comités électrotechniques nationaux (Comités nationaux de l'IEC). L'IEC a pour objet de favoriser la coopération internationale pour toutes les questions de normalisation dans les domaines de l'électricité et de l'électronique. A cet effet, l'IEC – entre autres activités – publie des Normes internationales, des Spécifications techniques, des Rapports techniques, des Spécifications accessibles au public (PAS) et des Guides (ci-après dénommés "Publication(s) de l'IEC"). Leur élaboration est confiée à des comités d'études, aux travaux desquels tout Comité national intéressé par le sujet traité peut participer. Les organisations internationales, gouvernementales et non gouvernementales, en liaison avec l'IEC, participent également aux travaux. L'IEC collabore étroitement avec l'Organisation Internationale de Normalisation (ISO), selon des conditions fixées par accord entre les deux organisations.
- 2) Les décisions ou accords officiels de l'IEC concernant les questions techniques représentent, dans la mesure du possible, un accord international sur les sujets étudiés, étant donné que les Comités nationaux de l'IEC intéressés sont représentés dans chaque comité d'études.
- 3) Les Publications de l'IEC se présentent sous la forme de recommandations internationales et sont agréées comme telles par les Comités nationaux de l'IEC. Tous les efforts raisonnables sont entrepris afin que l'IEC s'assure de l'exactitude du contenu technique de ses publications; l'IEC ne peut pas être tenue responsable de l'éventuelle mauvaise utilisation ou interprétation qui en est faite par un quelconque utilisateur final.
- 4) Dans le but d'encourager l'uniformité internationale, les Comités nationaux de l'IEC s'engagent, dans toute la mesure possible, à appliquer de façon transparente les Publications de l'IEC dans leurs publications nationales et régionales. Toutes divergences entre toutes Publications de l'IEC et toutes publications nationales ou régionales correspondantes doivent être indiquées en termes clairs dans ces dernières.
- 5) L'IEC elle-même ne fournit aucune attestation de conformité. Des organismes de certification indépendants fournissent des services d'évaluation de conformité et, dans certains secteurs, accèdent aux marques de conformité de l'IEC. L'IEC n'est responsable d'aucun des services effectués par les organismes de certification indépendants.
- 6) Tous les utilisateurs doivent s'assurer qu'ils sont en possession de la dernière édition de cette publication.
- 7) Aucune responsabilité ne doit être imputée à l'IEC, à ses administrateurs, employés, auxiliaires ou mandataires, y compris ses experts particuliers et les membres de ses comités d'études et des Comités nationaux de l'IEC, pour tout préjudice causé en cas de dommages corporels et matériels, ou de tout autre dommage de quelque nature que ce soit, directe ou indirecte, ou pour supporter les coûts (y compris les frais de justice) et les dépenses découlant de la publication ou de l'utilisation de cette Publication de l'IEC ou de toute autre Publication de l'IEC, ou au crédit qui lui est accordé.
- 8) L'attention est attirée sur les références normatives citées dans cette publication. L'utilisation de publications référencées est obligatoire pour une application correcte de la présente publication.
- 9) L'attention est attirée sur le fait que certains des éléments de la présente Publication de l'IEC peuvent faire l'objet de droits de brevet. L'IEC ne saurait être tenue pour responsable de ne pas avoir identifié de tels droits de brevets et de ne pas avoir signalé leur existence.

La Norme internationale IEC 62541-8 a été établie par le sous-comité 65E: Les dispositifs et leur intégration dans les systèmes de l'entreprise, du comité d'études 65 de l'IEC: Mesure, commande et automation dans les processus industriels.

Cette deuxième édition annule et remplace la première édition parue en 2011. Cette édition constitue une révision technique.

Cette édition inclut les modifications techniques majeures suivantes par rapport à l'édition précédente:

- a) Clarification précisant qu'il faut que la bande morte soit comprise entre 0,0 et 100,0. Les violations génèrent l'erreur `Bad_DeadbandFilterInvalid` (6.2)
- b) Ajout de `VariableTypes` (*Types de Variables*) gérant les `ArrayItems` (éléments de matrices) et les `DataTypes` (*Types de Données*) les prenant en charge, y compris les types de

nombre complexes. Ces types de données sont exigés pour les dispositifs d'analyses complexes mais semblent également utiles.

Le texte de cette norme est issu des documents suivants:

CDV	Rapport de vote
65E/381/CDV	65E/407/RVD

Le rapport de vote indiqué dans le tableau ci-dessus donne toute information sur le vote ayant abouti à l'approbation de cette norme.

Cette publication a été rédigée selon les Directives ISO/IEC, Partie 2.

Une liste de toutes les parties de la série IEC 62541, publiées sous le titre général *Architecture unifiée OPC*, peut être consultée sur le site web de l'IEC.

Le comité a décidé que le contenu de cette publication ne sera pas modifié avant la date de stabilité indiquée sur le site web de l'IEC sous "<http://webstore.iec.ch>" dans les données relatives à la publication recherchée. A cette date, la publication sera

- reconduite,
- supprimée,
- remplacée par une édition révisée, ou
- amendée.

IMPORTANT – Le logo "*colour inside*" qui se trouve sur la page de couverture de cette publication indique qu'elle contient des couleurs qui sont considérées comme utiles à une bonne compréhension de son contenu. Les utilisateurs devraient, par conséquent, imprimer cette publication en utilisant une imprimante couleur.

ARCHITECTURE UNIFIÉE OPC –

Partie 8: Accès aux données

1 Domaine d'application

La présente partie de l'IEC 62541 fait partie intégrante de la série de normes générales sur l'Architecture Unifiée OPC (OPC UA). Elle définit le modèle d'informations associé à l'Accès aux Données (DA). Elle comporte notamment des *Types de Variables (VariableTypes)* supplémentaires et des descriptions complémentaires des *Classes de Nœuds (NodeClasses)* et des *Attributs (Attributes)*, nécessaires pour l'Accès aux Données, des *Propriétés (Properties)* supplémentaires ainsi que d'autres paramètres relatifs aux informations et au comportement.

Le modèle d'Espace d'adresses complet, comprenant toutes les *NodeClasses* et tous les *Attributes* est spécifié dans l'IEC 62541-3. Les services permettant de détecter et d'accéder aux données sont spécifiés dans l'IEC 62541-4.

2 Références normatives

Les documents suivants sont cités en référence de manière normative, en intégralité ou en partie, dans le présent document et sont indispensables pour son application. Pour les références datées, seule l'édition citée s'applique. Pour les références non datées, la dernière édition du document de référence s'applique (y compris les éventuels amendements).

IEC TR 62541-1, *OPC Unified Architecture - Part 1: Overview and Concepts* (disponible en anglais seulement)

IEC 62541-3, *Architecture unifiée OPC - Partie 3: Modèle de l'espace d'adressage*

IEC 62541-4, *Architecture unifiée OPC - Partie 4: Services*

IEC 62541-5, *Architecture unifiée OPC - Partie 5: Modèle d'Information*

UN/CEFACT: **Recommandation UNECE N°. 20** – Codes pour les Unités de mesure utilisés dans le commerce international, disponible à http://www.unece.org/cefact/recommendations/rec_index.htm

3 Termes, définitions et abréviations

3.1 Termes et définitions

Pour les besoins du présent document, les termes et définitions donnés dans l'IEC TR 62541-1, l'IEC 62541-3 et l'IEC 62541-4, ainsi que les suivants s'appliquent.

3.1.1

Élément de données (DataItem)

liaison à des données arbitraires et réelles d'automatisation, c'est-à-dire des données qui représentent des informations en cours de validité

Note 1 à l'article: Exemples de ce type de données:

- données relatives aux appareils (tels que des capteurs de température),

- données calculées,
- informations d'état (ouvert/fermé, en déplacement),
- données du système en modification dynamique (telles que les prix en bourse),
- données de diagnostic.

3.1.2

Élément analogique (AnalogItem)

éléments de Données qui représentent des grandeurs physiques continuellement variables (par exemple, longueur, température), par opposition à la représentation numérique des données des éléments discrets

Note 1 à l'article: Des exemples types sont les valeurs fournies par des capteurs de température ou de pression. L'OPC UA définit un *Type de Variable* particulier permettant d'identifier un *Élément Analogique*. Les *Propriétés* décrivent les plages possibles des *Éléments Analogiques*.

3.1.3

Élément discret (DiscreteItem)

éléments de Données qui représentent des données qui ne peuvent avoir qu'un certain nombre de valeurs possibles (par exemple, OUVERTURE (OPENING), OUVERT (OPEN), FERMETURE (CLOSING), FERME (CLOSED))

Note 1 à l'article: Des *Types de Variables* particuliers sont utilisés pour identifier les *Éléments Discrets* à deux états ou à états multiples. Les *Propriétés* spécifient les valeurs de chaîne de ces états.

3.1.4

Élément de matrice (ArrayItem)

éléments de Données qui représentent des grandeurs physiques continuellement variables et où chaque point de données individuel comprend plusieurs valeurs représentées par une matrice (par exemple, la réponse spectrale d'un filtre numérique)

Note 1 à l'article: Des exemples types sont les données fournies par les dispositifs d'analyse. Les *Types de Variables* particuliers servent à identifier les variantes d'*Éléments de Matrices*.

3.1.5

Unités techniques (EngineeringUnits)

unités de mesure des *Éléments Analogiques* qui représentent des grandeurs physiques continuellement variables (par exemple, longueur, masse, temps, température)

Note 1 à l'article: La présente norme définit des *Propriétés* destinées à indiquer l'unité utilisée pour la valeur de l'*Élément de Données* et la valeur la plus haute et la plus basse susceptible d'être obtenue en fonctionnement normal.

3.2 Abréviations et symboles

DA	Data Access (Accès aux Données)
EU	Engineering Unit (Unité Technique)
UA	Unified Architecture (Architecture Unifiée)

4 Concepts

L'Accès aux Données concerne la représentation et l'utilisation des données d'automatisation dans les Serveurs-

Les données d'automatisation peuvent être situées dans le *Serveur* ou sur des cartes E/S directement connectées au *Serveur*. Elles peuvent également être situées dans des sous-serveurs ou sur d'autres appareils tels que des contrôleurs et des modules d'entrée/sortie, connectés par des liaisons série par l'intermédiaire de bus de terrain ou d'autres supports de communication. Les *Serveurs* OPC UA d'Accès aux Données fournissent aux *Clients* OPC UA un ou plusieurs Accès aux Données en offrant un accès transparent à leurs données d'automatisation.

Les liaisons aux instances de données d'automatisation sont désignées *Éléments de Données*. Les catégories de données d'automatisation fournies sont complètement spécifiques au fournisseur. La Figure 1 illustre la manière dont l'*Espace d'adresses (AddressSpace)* d'un *Serveur* peut comporter une large plage de différents *Éléments de Données*.

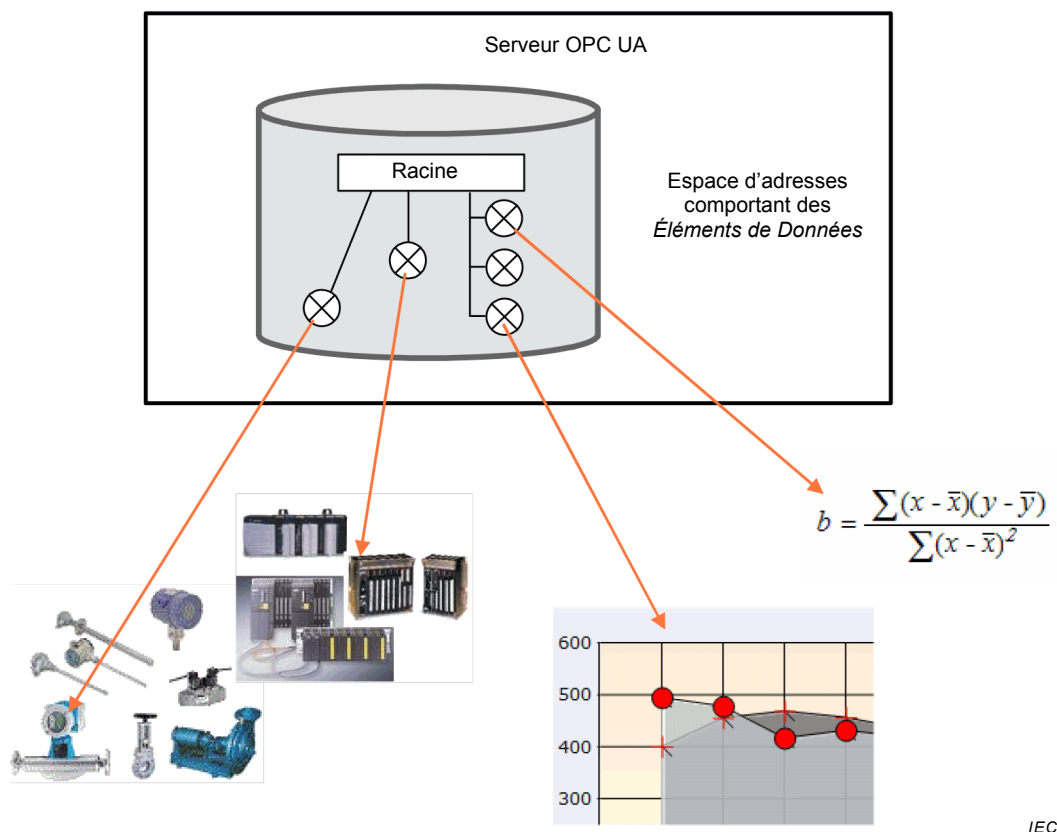


Figure 1 – Éléments de Données OPC reliés aux données d'automatisation

Les *Clients* peuvent lire ou écrire les *Éléments de Données* ou surveiller la modification des valeurs. Les *Services* nécessaires à ces opérations sont spécifiés dans l'IEC 62541-4. Les modifications sont définies comme une modification d'état (qualité) ou une modification de valeur qui dépasse une plage définie par le client désignée *Bande morte (DeadBand)*. Afin de pouvoir détecter une modification de valeur, la différence entre la valeur courante et la dernière valeur récupérée est comparée à la *Bande morte*.

5 Modèle

5.1 Généralités

Le modèle d'Accès aux Données étend le modèle de variable en définissant des *Types de Variables*. Le *Type d'Élément de Données (DataItem Type)* est le type de base. Le *Type d'Élément de Matrice (ArrayItem Type)*, le *Type d'Élément Analogique* et le *Type d'Élément Discret* (et ses sous-types *Deux États (TwoState)* et *États Multiples (Multistate)*) sont des spécialisations. Voir la Figure 2. Chacun de ces *Types de Variables* peut être davantage étendu pour former des *Éléments de Données* spécifiques au domaine ou au serveur.

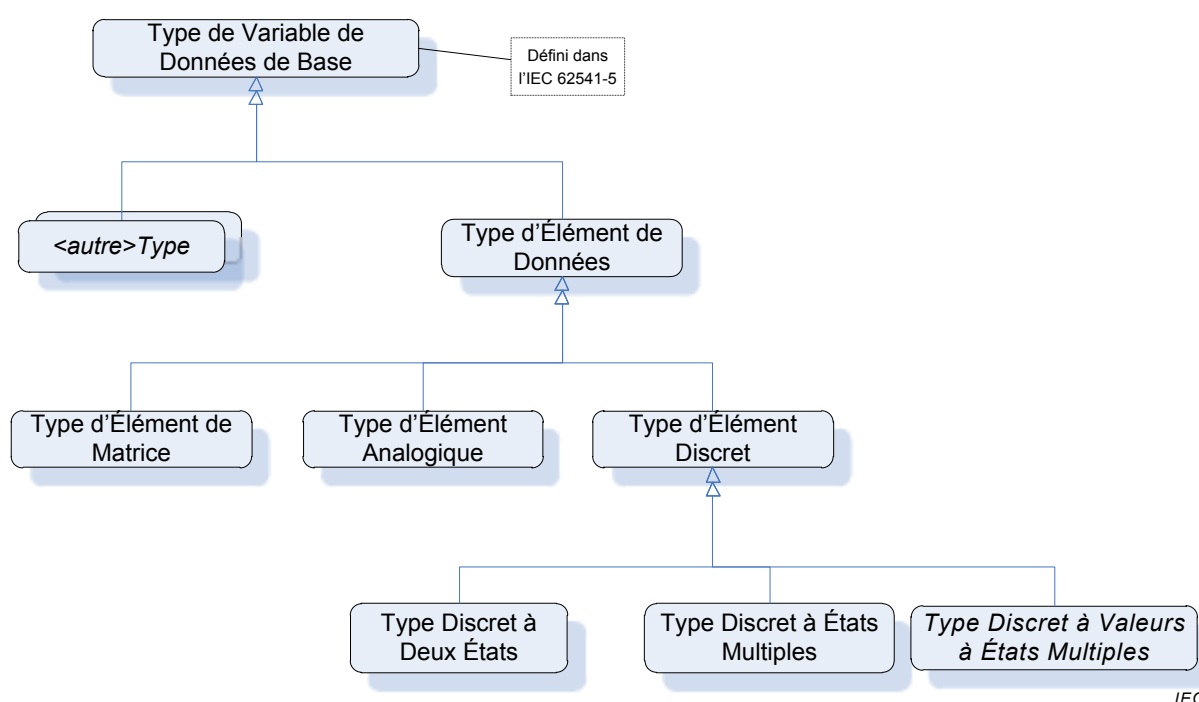


Figure 2 – Hiérarchie du Type de Variable *Élément de Données*

5.2 Modifications Sémantiques (SemanticsChanged)

Le *Code de Statut* (*StatusCode*) contient également un bit d'information appelé *Modifications Sémantiques*.

Les *Serveurs* qui mettent en œuvre l'Accès aux Données doivent définir ce Bit dans les notifications en cas de modification de certaines *Propriétés* définies dans la présente norme. Les *Propriétés* correspondantes sont spécifiées de manière individuelle pour chaque *Type de Variable*.

Il convient que les *Clients* qui utilisent l'une quelconque de ces *Propriétés* les relisent avant de traiter la valeur de données.

5.3 Types de Variables

5.3.1 Type d'Élément de Données (DataItem Type)

Ce *Type de Variable* définit les caractéristiques générales d'un *Élément de Données*. Tous les autres *Types d'Éléments de Données* dérivent de ce type de variable. Le *Type d'Élément de Données* dérive du *Type de Variable de Données de Base* (*BaseDataVariableType*) et partage par conséquent le modèle de variable, tel que décrit dans l'IEC 62541-3 et l'IEC 62541-5. Il est défini de façon formelle dans le Tableau 1.

Tableau 1 – Définition du Type d'Élément de Données

Attribut	Valeur				
BrowseName	Type d'Élément de Données				
IsAbstract	False				
ValueRank	-2 (-2 = 'Tout')				
DataType	Type de Données de Base				
Références	NodeClass	BrowseName	DataType	TypeDefinition	Règle de Modélisation
Sous-type du <i>Type de Variable de Données de Base</i> défini dans l'IEC 62541-5, c'est-à-dire les <i>Propriétés</i> de ce type sont héritées.					
HasSubtype(	Type de Variable	Type d'Élément Analogique	Défini en 5.3.2		
HasSubtype	Type de Variable	Type d'Élément Discret	Défini en 5.3.3		
HasSubtype	Type de Variable	Type d'Élément de Matrice	Défini en 5.3.4		
HasProperty	Variable	Définition	Chaîne	Type de Propriété	Facultative
HasProperty	Variable	Précision de la Valeur	Double	Type de Propriété	Facultative

La *Définition (Definition)* est une chaîne spécifique au fournisseur et interprétable par l'utilisateur qui définit le mode de calcul de la valeur de cet *Élément de Données*. La *Définition* est non localisée et contient le plus souvent une équation qui peut être analysée par certains clients.

Exemple: *Definition::= "(TempA - 25) + TempB"*

La *Précision de la Valeur (ValuePrecision)* spécifie la précision maximale que le *Serveur* peut maintenir pour l'élément sur la base de restrictions dans l'environnement cible.

La *Précision de la Valeur* peut être utilisée pour les *Types de Données* suivants:

- Pour les valeurs à Virgule Flottante et à précision Double, elle spécifie le nombre de chiffres après la virgule.
- Pour les valeurs Date&Heure, elle indique l'intervalle de temps minimal en nanosecondes. Par exemple, une Précision de la Valeur de 20 000 000 définit une précision de 20 ms.

La *Propriété Précision de la Valeur* est une approximation destinée à servir de lignes directrices pour le *Client*. Un *Serveur* est supposé arrondir discrètement toute valeur qu'il prend en charge avec plus de précision. Ceci implique qu'un *Client* peut être confronté à une situation dans laquelle la valeur relue et renvoyée par le *Serveur* est différente de celle qu'il a écrite et envoyée au *Serveur*. Cette différence ne doit pas être supérieure à celle proposée par cette *Propriété*.

5.3.2 Type d'Élément Analogique (AnalogItemType)

Ce *Type de Variable* définit les caractéristiques générales d'un *Élément Analogique*. Tous les autres *Types d'Éléments Analogiques* dérivent de ce type de variable. Le *Type d'Élément Analogique* dérive du *Type d'Élément de Données*. Il est défini de façon formelle dans le Tableau 2.

Tableau 2 – Définition du Type d'Élément Analogique

Attribut	Valeur				
BrowseName	Type d'Élément Analogique				
IsAbstract	False				
ValueRank	-2 (-2 = 'Tout')				
DataType	Nombre				
Références	NodeClass	BrowseName	DataType	TypeDefinition	Règle de Modélisation
Sous-type du <i>Type d'Élément de Données</i> défini en 5.3.1, c'est-à-dire les <i>Propriétés</i> de ce type sont héritées.					
HasProperty	Variable	Étendue de Mesure	Plage	Type de Propriété	Facultative
HasProperty	Variable	Plage EU	Plage	Type de Propriété	Obligatoire
HasProperty	Variable	Unités Techniques	Information EU	Type de Propriété	Facultative

Les alinéas suivants décrivent les *Propriétés* de ce *Type de Variable*. Si la *Valeur* de l'élément analogique contient une matrice, les *Propriétés* doivent s'appliquer à tous les éléments de la matrice.

L'*Étendue de Mesure (InstrumentRange)* définit la plage de valeur que l'instrument peut renvoyer.

Exemple: `InstrumentRange ::= {-9999.9, 9999.9}`

Bien que cela soit défini comme facultatif, il est vivement recommandé que les *Serveurs* prennent en charge cette *Propriété*. En l'absence d'*Étendue de Mesure*, les *Clients* utiliseront généralement la plage complète selon le *Type de Données*.

Le *Type de Données Plage (Range Data Type)* est spécifié en 5.6.2.

La *Plage EU (EURange)* définit la plage de valeurs susceptible d'être obtenue en fonctionnement normal. Elle est destinée à être utilisée comme mise à l'échelle automatique d'un affichage de diagramme à barres.

La défaillance ou la désactivation d'un capteur ou d'un instrument peut donner lieu au renvoi d'une valeur d'élément qui est en réalité en dehors de cette plage. Il faut que le logiciel client soit préparé à pouvoir gérer cette possibilité. De même, un *Client* peut tenter d'écrire une valeur qui est en dehors de la plage de renvoi au serveur. Dans ce cas, le comportement exact (accepter, refuser, retenir, etc.) dépend du *Serveur*. Cependant, les *Serveurs* doivent normalement être préparés à gérer cette situation.

Exemple: `EURange ::= {-200.0, 1400.0}`

Voir également 6.2 concernant un filtre de surveillance spécial (*Pourcentage de Bande morte*)(*PercentDeadband*) qui est basé sur la plage d'unité technique.

Les *Unités Techniques (EngineeringUnits)* spécifient les unités de la valeur de l'*Élément de Données* (par exemple, DEGC, hertz, secondes). Le type d'*information EU (EUInformation)* est spécifié en 5.6.3.

Note importante: Comprendre les unités d'une valeur de mesure est essentiel pour un système uniforme. Dans un système ouvert, notamment où des serveurs issus de différents horizons pourraient être utilisés, il est primordial de connaître les unités de mesure. Sur la base de ces connaissances, les valeurs peuvent être converties si nécessaire avant leur utilisation. Par conséquent, bien qu'elle soit définie comme facultative, la prise en charge de la *Propriété Unités Techniques* est vivement recommandée.

L'OPC UA recommande d'utiliser les "Codes for Units of Measurement" (**Codes pour les Unités de mesure**) (voir (voir UN/CEFACT: **Recommandation UNECE N°. 20**)). La correspondance avec la *Propriété Unités Techniques* est spécifiée en 5.6.3.

EXEMPLE DE CONFUSION D'UNITES: En 1999, le vaisseau spatial Mars Climate Orbiter s'est écrasé sur la surface de la planète Mars. Une divergence concernant les unités utilisées en fut la raison principale. Le logiciel de navigation attendait des données en newton-seconde, tandis que la société qui avait construit le véhicule orbital avait fourni des données en livres-force par seconde. On peut évoquer une autre déconvenue, cependant moins coûteuse, lorsque les clients habitués aux pintes britanniques commandent une pinte aux États-Unis, et qu'on leur sert simplement ce qu'ils considèrent comme une «tromperie sur la marchandise».

Le bit de *Modifications Sémantiques du Code de Statut* (*StatusCode SemanticsChanged*) doit être défini en cas de modification de l'une quelconque des *Propriétés Plage EU* (pourrait modifier le comportement d'un *Abonnement* (*Subscription*) en cas d'utilisation d'un filtre à *Pourcentage de Bande morte*) ou des *Unités techniques* (pourrait générer des problèmes si le client utilise la valeur pour effectuer des calculs) (voir 5.2 pour des informations supplémentaires).

5.3.3 Type d'Élément Discret (*DiscreteItemType*)

5.3.3.1 Généralités

Ce Type de Variable est un type abstrait. En d'autres termes, aucune instance de ce type ne peut exister. Cependant, il peut être utilisé dans un filtre lors de la navigation ou de l'interrogation. Le *Type d'Élément Discret* dérive du *Type d'Élément de Données* et partage par conséquent toutes ses caractéristiques. Il est défini de façon formelle dans le Tableau 3.

Tableau 3 – Définition du Type d'Élément Discret

Attribut	Valeur				
BrowseName	Type d'Élément Discret				
IsAbstract	True				
ValueRank	-2 (-2 = 'Tout')				
DataType	Type de Données de Base				
Références	NodeClass	BrowseName	DataType	TypeDefinition	Règle de Modélisation
Sous-type du <i>Type d'Élément de Données</i> défini en 5.3, c'est-à-dire les <i>Propriétés</i> de ce type sont héritées.					
HasSubtype	Type de Variable	Type Discret à Deux États	Défini en 5.3.3.2		
HasSubtype	Type de Variable	Type Discret à États Multiples	Défini en 5.3.3.3		
HasSubtype	Type de Variable	Type Discret à Valeurs à États Multiples	Défini en 5.3.3.4		

5.3.3.2 Type Discret à Deux États (*TwoStateDiscreteType*)

Ce *Type de Variable* définit les caractéristiques générales d'un *Élément Discret* qui peut avoir deux états. Le *Type Discret à Deux États* dérive du *Type d'Élément Discret*. Il est défini de façon formelle dans le Tableau 4.

Tableau 4– Définition du Type Discret à Deux États

Attribut	Valeur				
BrowseName	Type Discret à Deux États				
IsAbstract	False				
ValueRank	-2 (-2 = 'Tout')				
DataType	Booléen				
Références	NodeClass	BrowseName	DataType	TypeDefinition	Règle de Modélisation
Sous-type du <i>Type d'Élément Discret</i> défini en 5.3.3, c'est-à-dire les <i>Propriétés</i> de ce type sont héritées.					
HasProperty	Variable	État True	Texte Localisé	Type de Propriété	Obligatoire
HasProperty	Variable	État False	Texte Localisé	Type de Propriété	Obligatoire

L'*État True* (*TrueState*) contient une chaîne à associer à cet *Élément de Données* lorsqu'il est TRUE. Ceci est généralement utilisé pour un contact lorsqu'il est dans l'état fermé (non nul).

Par exemple "EN MARCHÉ", "FERME", "ACTIF", "SECURISE" ("RUN", "CLOSE", "ENABLE", "SAFE"), etc.

L'*État False* (*FalseState*) contient une chaîne à associer à cet *Élément de Données* lorsqu'il est FALSE. Ceci est généralement utilisé pour un contact lorsqu'il est dans l'état ouvert (nul).

Par exemple "ARRÊT", "OUVERT", "INACTIF", "NON SECURISE" ("STOP", "OPEN", "DISABLE", "UNSAFE"), etc.

Si l'élément contient une matrice, les *Propriétés* s'appliquent alors à tous les éléments de la matrice.

Le bit de *Modifications Sémantiques du Code de Statut* (*StatusCode SemanticsChanged*) doit être défini en cas de modification de l'une quelconque des *Propriétés Etat False* ou *Etat True* (modifications pouvant entraîner une mauvaise interprétation de la part des utilisateurs ou des programmes (script)) (voir 5.2 pour des informations supplémentaires).

5.3.3.3 Type Discret à États Multiples (*MultiStateDiscreteType*)

Ce *Type de Variable* définit les caractéristiques générales d'un *Élément Discret* qui peut avoir plus de deux états. Le *Type Discret à États Multiples* dérive du *Type d'Élément Discret*. Il est défini de façon formelle dans le Tableau 5.

Tableau 5 – Définition du Type Discret à États Multiples

Attribut	Valeur				
BrowseName	Type Discret à États Multiples				
IsAbstract	False				
ValueRank	-2 (-2 = 'Tout')				
DataType	Entier Non Signé				
Références	NodeClass	BrowseName	DataType	TypeDefinition	Règle de Modélisation
Sous-type du <i>Type d'Élément Discret</i> défini en 5.3.3, c'est-à-dire les <i>Propriétés</i> de ce type sont héritées.					
HasProperty	Variable	Chaînes d'Énumération (<i>EnumStrings</i>)	Texte Localisé []	Type de Propriété	Obligatoire

La *Chaîne d'Énumération* (*EnumStrings*) est une table de consultation de chaîne correspondant aux valeurs numériques séquentielles (0, 1, 2, etc.)

Exemple:

"OUVERT"

"FERME"

"EN TRANSIT" ("IN TRANSIT"), etc.

Dans ce cas, la chaîne "OUVERT" correspond à 0, "FERME" à 1 et "EN TRANSIT" à 2.

Il convient que les clients soient préparés à gérer les valeurs d'éléments en dehors de la plage de la liste. Il convient que les serveurs robustes soient préparés à gérer les écritures de valeurs invalides.

Si l'élément contient une matrice, cette table de consultation doit alors s'appliquer à tous les éléments de la matrice.

NOTE La propriété *Chaînes d'Énumération* est également utilisée pour les *Types de Données Énumération* (pour la spécification de ce *Type de Données*, voir IEC 62541-3).

Le bit de *Modifications Sémantiques du Code de Statut* doit être défini en cas de modification de la *Propriété Chaînes d'Énumération (EnumStrings)* (des modifications peuvent entraîner une mauvaise interprétation par les utilisateurs ou les programmes (de script) (voir 5.2 pour des informations supplémentaires).

5.3.3.4 Type Discret à Valeurs à États Multiples (MultiStateValueDiscreteType)

Ce *Type de Variable* définit les caractéristiques générales d'un *Élément Discret* qui peut comporter deux états ou plus et où les valeurs d'état (l'énumération) ne consistent pas en des valeurs numériques consécutives (il peut y avoir des espaces), ou avec lesquels l'énumération n'est pas à base zéro. Le *Type Discret à Valeurs à États Multiples* dérive du *Type d'Élément Discret*. Il est défini de façon formelle dans le Tableau 6.

Tableau 6 – Définition du Type Discret à Valeurs à États Multiples

Attribut	Valeur				
BrowseName	Type Discret à Valeurs à États Multiples				
IsAbstract	False				
ValueRank	Scalaire				
DataType	Nombre				
Références	NodeClass	BrowseName	DataType	TypeDefinition	Règle de modélisation
Sous-type du <i>Type d'Élément Discret</i> défini en 5.3.3; c'est-à-dire que les <i>Propriétés</i> de ce type sont héritées.					
HasProperty	Variable	Valeurs d'énumération (EnumValues)	Voir IEC 62541-3		Obligatoire
HasProperty	Variable	Valeur sous forme de Texte (ValueAsText)	Voir IEC 62541-3		Obligatoire

Les *Valeurs d'Énumération* constituent une matrice du *Type de Valeur d'Énumération (EnumValueType)*. Chaque entrée de la matrice représente une valeur d'énumération avec sa notation sous forme d'entiers, une représentation interprétable par l'utilisateur et des informations d'aide. Ceci représente des énumérations avec des entiers qui ne sont pas à base zéro ou comportent des espaces (par exemple, 1, 2, 4, 8, 16). Voir l'IEC 62541-3 pour la définition de ce type. Les *Variables Discrètes à Valeurs à États Multiples* présentent la notation sous forme d'entiers actuelle dans leur *Attribut Valeur (Value Attribute)*. Les *Clients* lisent souvent la *Propriété Valeurs d'Énumération* à l'avance et la masquent pour consulter un nom ou une aide à chaque fois qu'ils reçoivent la représentation numérique.

Les *Variables Discrètes à Valeurs à États Multiples* peuvent comporter tout *Type de données* numériques; ceci inclut les entiers signés et non signés d'une longueur de 8 bits à 64 bits.

La représentation numérique de la valeur d'énumération courante est fournie via l'*Attribut Valeur* de la *Variable Discrète à Valeurs à États Multiples*. La *Propriété Valeur sous forme de Texte (ValueAsText Property)* fournit la représentation de texte localisé de la valeur d'énumération. Elle peut être utilisée par les *Clients* qui s'intéressent uniquement à l'affichage du texte d'abonnement à la *Propriété* en lieu et place de l'*Attribut Valeur*.

5.3.4 Type d'élément de matrice (ArrayItemType)

5.3.4.1 Généralités

Ce *Type de Variable* abstrait définit les caractéristiques générales d'un *Élément de matrice*. Les valeurs sont présentées dans une matrice, mais le contenu de cette matrice représente une entité unique telle qu'une image. Les autres *Éléments de Données* peuvent contenir les matrices qui représentent, par exemple, plusieurs valeurs de plusieurs capteurs de température d'une chaudière.

Le *Type d'Élément de Matrice* ou son sous-type doit être utilisé uniquement lorsque les *Propriétés Titre* et *Type de Facteur d'échelle selon un Axe* (*Title and AxisScaleType Properties*) peuvent être remplies avec des valeurs justifiées. Lorsque tel n'est pas le cas, le *Type d'Élément de Données* et les sous-types tels que le *Type d'Élément analogique*, qui prennent également en charge les matrices, doivent être utilisés. Le *Type d'Élément de Matrice* est défini de façon formelle dans le Tableau 7.

Tableau 7 – Définition du Type d'Élément de Matrice

Attribut	Valeur				
BrowseName	Type d'Élément de Matrice				
IsAbstract	True				
ValueRank	0 (0 = Une ou plusieurs Dimensions (OneOrMoreDimensions))				
DataType	Type de Données de Base				
Références	NodeClass	BrowseName	DataType	TypeDefinition	Règle de modélisation
Sous-type du <i>Type d'Élément de Données</i> défini en 5.3.1; c'est-à-dire que les <i>Propriétés</i> de ce type sont héritées.					
HasSubtype	Type de Variable	Type d'Élément de Matrice Y (YarrayItemType)	Défini en 5.3.4.2		
HasSubtype	Type de Variable	Type d'Élément de Matrice XY (XYArrayItemType)	Défini en 5.3.4.3		
HasSubtype	Type de Variable	Type d'Élément d'Image (ImageItemType)	Défini en 5.3.4.4		
HasSubtype	Type de Variable	Type d'Élément de Cube (CubeItemType)	Défini en 5.3.4.5		
HasSubtype	Type de Variable	Type d'Élément de Matrice de Dimension N (NdimensionArrayItemType)	Défini en 5.3.4.6		
HasProperty	Variable	Étendue de mesure	Plage	Type de propriété	Facultative
HasProperty	Variable	Plage EU	Plage	Type de propriété	Obligatoire
HasProperty	Variable	Unités techniques	Information EU	Type de propriété	Obligatoire
HasProperty	Variable	Titre (Title)	Texte localisé	Type de propriété	Obligatoire
HasProperty	Variable	Type de Facteur d'échelle selon un Axe (AxisScaleType)	Énumération de facteur d'échelle selon un axe (AxisScaleEnumeration)	Type de propriété	Obligatoire

L'*Étendue de mesure* définit la plage de la *Valeur* de l'*Élément de Matrice*.

La *Plage EU* définit la plage de valeurs de l'*Élément de Matrice* susceptible d'être obtenu en fonctionnement normal. Elle est destinée à une utilisation telle que la mise à l'échelle automatique d'une représentation de diagramme à barres.

Les *Unités Techniques* comportent des informations sur les unités techniques de la *Valeur* de l'*Élément de Matrice*.

Pour les informations supplémentaires concernant l'*Étendue de mesure*, la *plage EU* et les *Unités Techniques*, voir la description du *Type d'Élément Analogique* en 5.3.2.

Le *Titre* comporte le titre lisible par l'utilisateur de la *Valeur* de l'*Élément de Matrice*.

Le *Type de Facteur d'échelle selon un Axe* définit l'échelle à utiliser pour l'axe où la *Valeur* de l'*Élément de Matrice* doit être affichée.

Le bit de *Modifications Sémantiques du Code de Statut* doit être défini en cas de modification de l'une quelconque des *Propriétés Étendue de mesure*, *Plage EU*, *Unités Techniques* ou *Titre* (voir 5.2 pour des informations supplémentaires).

5.3.4.2 Type d'Élément de Matrice Y (YArrayItemType)

Le *Type d'Élément de Matrice Y* représente une matrice unidimensionnelle de valeurs numériques utilisée pour représenter des spectres ou des distributions où les intervalles de l'axe des x (abscisse) sont constants. Le *Type d'Élément de Matrice Y* est défini de façon formelle dans le Tableau 8.

Tableau 8 – Définition du Type d'Élément de Matrice Y

Attribut	Valeur				
BrowseName	Type d'Élément de Matrice Y				
IsAbstract	False				
ValueRank	1				
DataType	Type de Données de Base				
ArrayDimensions	{0} (0 = Taille inconnue (UnknownSize))				
Références	NodeClass	BrowseName	DataType	TypeDefinition	Règle de modélisation
Sous-type du <i>Type d'Élément de Matrice</i> défini en 5.3.4.1					
HasProperty	Variable	Définition de l'axe des X (abscisse) (XaxisDefinition)	Informations concernant l'axe (AxisInformation)	Type de propriété (PropertyType)	Obligatoire

La *Valeur* de l'*Élément de Matrice Y* contient les valeurs numériques pour l'axe des Y (Axe des ordonnées). Les *Unités techniques* et la *Plage* de la *Valeur* sont définies par les *Propriétés* correspondantes héritées du *Type d'Élément de Matrice*.

Le *Type de Données* de ce *Type de Variable* est limité aux SOctet (Sbyte), Int16, Int32, Int64, Virgule flottante (Float), Double (Double), Type de Nombre Complexe (ComplexNumberType) et Type de Nombre Complexe Double (DoubleComplexNumberType).

La *Propriété Définition de l'Axe des X* (XAxisDefinition Property) contient des informations sur les *Unités Techniques* et la *Plage* relative à l'axe des X.

Le bit de *Modifications Sémantiques du Code de Statut* doit être défini en cas de modification de l'une quelconque des cinq *Propriétés* suivantes: *Étendue de mesure*, *Plage EU*, *Unités Techniques*, *Titre* ou *Définition de l'axe des X* (voir 5.2 pour des informations supplémentaires).

La Figure 3 présente un exemple de la façon dont les *Attributs* et les *Propriétés* peuvent être utilisés dans une interface graphique.

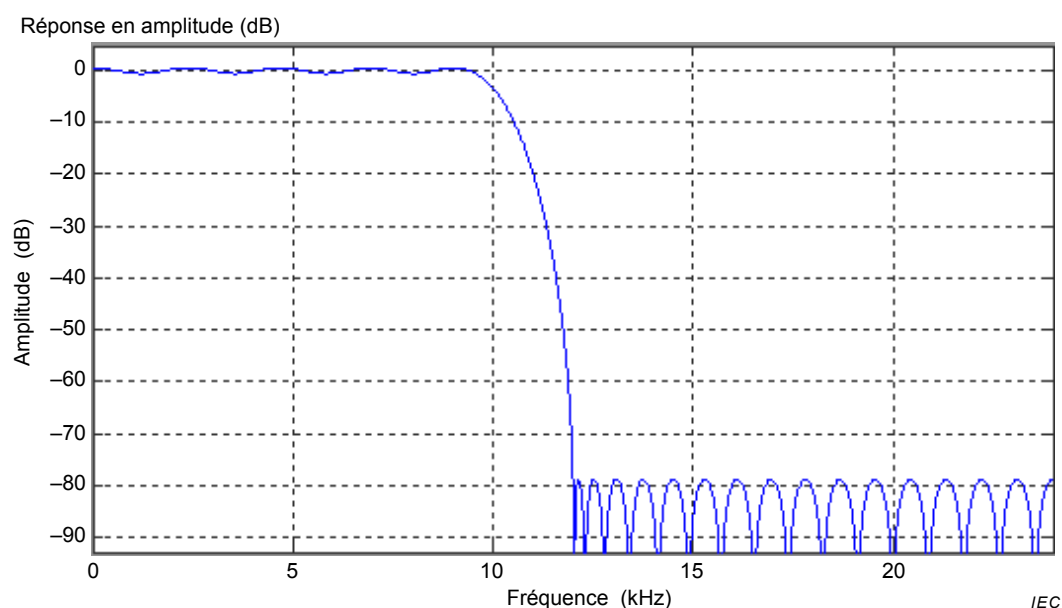


Figure 3 – Représentation graphique d'un Élément de Matrice Y

Le Tableau 9 décrit les valeurs de chaque élément présenté à la Figure 3.

Tableau 9 – Description de l'Élément de Matrice Y

Attribut / Propriété	Valeur d'élément
Description	Réponse en amplitude (dB)
Type de Facteur d'Échelle selon un Axe	AxisScaleEnumeration.LINEAR_0
Étendue de mesure.Bas. (InstrumentRange.low)	-90
Étendue de mesure.Haut. (InstrumentRange.high)	5
Plage EU.bas (EURange.low)	-90
Plage EU.haut (EURange.high)	2
Unités Techniques.Url d'espace de nom (EngineeringUnits.namespaceUrl)	http://www.opcfoundation.org/UA/units/un/cefact
Unités Techniques.Id d'unité (EngineeringUnits.unitId)	2N
Unités Techniques.Nom d'affichage (EngineeringUnits.displayName)	"en-us", "dB"
Unités Techniques.description (EngineeringUnits.description)	"en-us", "decibel"
Titre	Amplitude
Définition de l'Axe des X.Unités Techniques.Url d'espace de nom (XAxisDefinition.EngineeringUnits.namespaceUrl)	http://www.opcfoundation.org/UA/units/un/cefact
Définition de l'Axe des X.Unités Techniques.Id d'unité (XAxisDefinition.EngineeringUnits.unitId)	kHz
Définition de l'Axe des X.Unités Techniques.nom d'affichage (XAxisDefinition.EngineeringUnits.displayName)	"en-us", "kHz"
Définition de l'Axe des X.Unités Techniques.description (XAxisDefinition.EngineeringUnits.description)	"en-us", "kilohertz"
Définition de l'Axe des X.Plage. bas (XAxisDefinition.Range.low)	0
Définition de l'Axe des X.Plage. haut (XAxisDefinition.Range.high)	25

Attribut / Propriété	Valeur d'élément
Définition de l'Axe des X.titre (XAxisDefinition.title)	"en-us", "Frequency"
Définition de l'Axe des X.Type de Facteur d'Échelle selon un Axe (XAxisDefinition.axisScaleType)	AxisScaleEnumeration.LINEAR_0
Définition de l'Axe des X.Paliers d'axe (XAxisDefinition.axisSteps)	null

Notes d'interprétation:

- Les éléments de ce tableau ne sont pas tous utilisés dans le graphique.
- L'axe des X est affiché selon un ordre inversé, cependant, la *Définition de l'Axe des X.Plage.bas* doit être inférieure à la *Définition de l'Axe des X.Plage.haut*. Seule une représentation graphique inverse l'ordre d'affichage.
- L'axe des X est constant

5.3.4.3 Type d'Élément de Matrice XY (XYArrayItemType)

Le *Type d'Élément de Matrice XY* représente un vecteur des valeurs de Type XV telles qu'une liste des valeurs de crête, où TypeXV.x (XVType.x) est la position de la valeur crête et TypeXV.valeur (XVType.value) est son intensité. Le *Type d'Élément de Matrice XY* est défini de façon formelle dans le Tableau 10.

Tableau 10 – Définition du Type d'Élément de Matrice XY

Attribut	Valeur				
BrowseName	Type d'Élément de Matrice XY				
IsAbstract	False				
ValueRank	1				
DataType	Type XV (défini en 5.6.8)				
Références	NodeClass	BrowseName	DataType	TypeDefinition	Règle de modélisation
Sous-type du <i>Type d'Élément de Matrice</i> défini en 5.3.4.1					
HasProperty	Variable	Définition de l'Axe des X	Informations concernant l'Axe	Type de propriété	Obligatoire

La *Valeur* de l'*Élément de Matrice XY* contient une matrice de structures (TypeXV) où chaque structure spécifie la position de l'axe des X (Type XV.x) et la valeur elle-même (TypeXV.valeur), utilisée pour l'Axe des Y. Les unités techniques et la plage applicable à la *Valeur* sont définies par les *Propriétés* correspondantes héritées du *Type d'Élément de Matrice*.

La *Propriété Définition de l'Axe des X* contient des informations sur les *Unités Techniques* et la *Plage* applicable à l'Axe des X.

Les *Paliers d'axe* de la *Définition de l'Axe des X* doivent être mis à NULL parce qu'ils ne sont pas utilisés.

Le bit de *Modifications Sémantiques du Code de Statut* doit être défini en cas de modification de l'une quelconque des *Propriétés Etendue de mesure*, *Plage EU*, *Unités techniques*, *Titre* ou *Définition de l'Axe des X* (voir 5.2 pour des informations supplémentaires).

5.3.4.4 Type d'Élément d'Image (ImageItemType)

Le *Type d'Élément d'Image* définit les caractéristiques générales d'un *Élément d'Image* qui représente une matrice de valeurs telle qu'une image, où la position des pixels est donnée par X qui correspond à la colonne et Y à la rangée. La valeur correspond à l'intensité des pixels.

Le *Type d'Élément d'Image* est défini de façon formelle dans le Tableau 11.

Tableau 11 – Définition du Type d'Élément d'Image

Attribut	Valeur				
BrowserName	Type d'Élément d'Image				
IsAbstract	False				
ValueRank	2 (2 = matrice à deux dimensions)				
DataType	Type de Données de Base				
Références	NodeClass	BrowserName	DataType	TypeDefinition	Règle de modélisation
Sous-type du <i>Type d'Élément de Matrice</i> défini en 5.3.4.1					
HasProperty	Variable	Définition de l'Axe des X	Informations concernant l'Axe	Type de propriété	Obligatoire
HasProperty	Variable	Définition de l'Axe des Y	Informations concernant l'Axe	Type de propriété	Obligatoire

Les unités techniques et la plage applicable à la *Valeur* sont définies par les *Propriétés* correspondantes héritées du *Type d'Élément de Matrice*.

Le *Type de Données* de ce *Type de Variable* est limité aux SOctet, Int16, Int32, Int64, Virgule flottante, Double, Type de Nombre Complexe et Type de Nombre Complexe Double.

L'*Attribut Dimensions de la Matrice (ArrayDimensions Attribute)* pour les *Variables* de ce type ou les sous-types doit utiliser la première entrée dans la matrice ([0]) pour définir le nombre de colonnes et la seconde entrée ([1]) pour définir le nombre de rangées, en supposant que la taille de la matrice n'est pas dynamique.

La *Propriété Définition de l'Axe des X* contient des informations sur les unités techniques et la plage applicable à l'axe des X.

La *Propriété Définition de l'Axe des Y* contient des informations sur les unités techniques et la plage applicable à l'axe des Y.

Le bit de *Modifications Sémantiques du Code de Statut* doit être défini en cas de modification de l'une quelconque des *Propriétés Etendue de mesure, Plage EU, Unités Techniques, Titre, Définition de l'Axe des X* ou *Définition de l'Axe des Y*.

5.3.4.5 Type d'Élément de Cube (CubeItem Type)

Le *Type d'Élément de Cube* représente un cube de valeurs tel qu'une distribution de particules dans l'espace, où la position des particules est donnée par X qui correspond à la colonne, Y à la rangée et Z à la profondeur. Dans l'exemple d'une distribution des particules dans l'espace, la valeur correspond à la granulométrie. Le *Type d'Élément de Cube* est défini de façon formelle dans le Tableau 12.

Tableau 12 – Définition du Type d'Élément de Cube

Attribut	Valeur				
BrowseName	Type d'Élément de Cube				
IsAbstract	False				
ValueRank	3 (3 = matrice à trois dimensions)				
DataType	Type de Données de Base				
Références	NodeClass	BrowseName	DataType	TypeDefinition	Règle de modélisation
Sous-type du <i>Type d'Élément de Matrice</i> défini en 5.3.4.1					
HasProperty	Variable	Définition de l'Axe des X	<i>Informations concernant l'Axe</i>	Type de propriété	Obligatoire
HasProperty	Variable	Définition de l'Axe des Y	<i>Informations concernant l'Axe</i>	Type de propriété	Obligatoire
HasProperty	Variable	Définition de l'Axe des Z	<i>Informations concernant l'Axe</i>	Type de propriété	Obligatoire

Les unités techniques et la plage applicable à la *Valeur* sont définies par les *Propriétés* correspondantes héritées du *Type d'Élément de Matrice*.

Le *Type de Données* de ce *Type de Variable* est limité aux SOctet, Int16, Int32, Int64, Virgule flottante, Double, *Type de Nombre Complexe* et *Type de Nombre Complexe Double*.

Il convient que l'*Attribut Dimensions de la Matrice (ArrayDimensions Attribute)* pour les *Variables* de ce type ou les sous-types utilise la première entrée dans la matrice ([0]) pour définir le nombre de colonnes, la deuxième entrée ([1]) pour définir le nombre de rangées et la troisième entrée ([2]) pour définir le nombre de paliers de l'axe des Z, en supposant que la taille de la matrice n'est pas dynamique.

La *Propriété Définition de l'Axe des X* contient des informations sur les unités techniques et la plage applicable à l'axe des X.

La *Propriété Définition de l'Axe des Y* contient des informations sur les unités techniques et la plage applicable à l'axe des Y.

La *Propriété Définition de l'Axe des Z* contient des informations sur les unités techniques et la plage applicable à l'axe des Z.

Le bit de *Modifications Sémantiques du Code de Statut* doit être défini en cas de modification de l'une quelconque des *Propriétés Etendue de mesure*, *Plage EU*, *Unités Techniques*, *Titre*, *Définition de l'Axe des X*, *Définition de l'Axe des Y* ou *Définition de l'Axe des Z* (voir 5.2 pour des informations supplémentaires).

5.3.4.6 Type d'Élément de Matrice à N Dimensions (NDimensionArrayItem Type)

Ce *Type de Variable* définit un *Élément de Matrice* à plusieurs dimensions.

Cette approche réduit au minimum le nombre de types; il peut toutefois se révéler plus difficile à utiliser pour les interactions de systèmes de commande.

Le *Type d'Élément de Matrice à N Dimensions* est défini de façon formelle dans le Tableau 13.

Tableau 13 – Définition du Type d'Élément de Matrice à N Dimensions

Attribut	Valeur				
BrowseName	Type d'Élément de Matrice à N dimensions				
IsAbstract	False				
ValueRank	0 (0 = Une ou plusieurs Dimensions)				
DataType	Type de Données de Base				
Références	NodeClass	BrowseName	DataType	TypeDefinition	Règle de modélisation
Sous-type du <i>Type d'Élément de Matrice</i> défini en 5.3.4.1					
HasProperty	Variable	Définition de l'Axe	Informations concernant l'Axe []	Type de propriété	Obligatoire

Le *Type de Données* de ce *Type de Variable* est limité aux SOctet, Int16, Int32, Int64, Virgule flottante, Double, Type de Nombre Complexe et Type de Nombre Complexe Double.

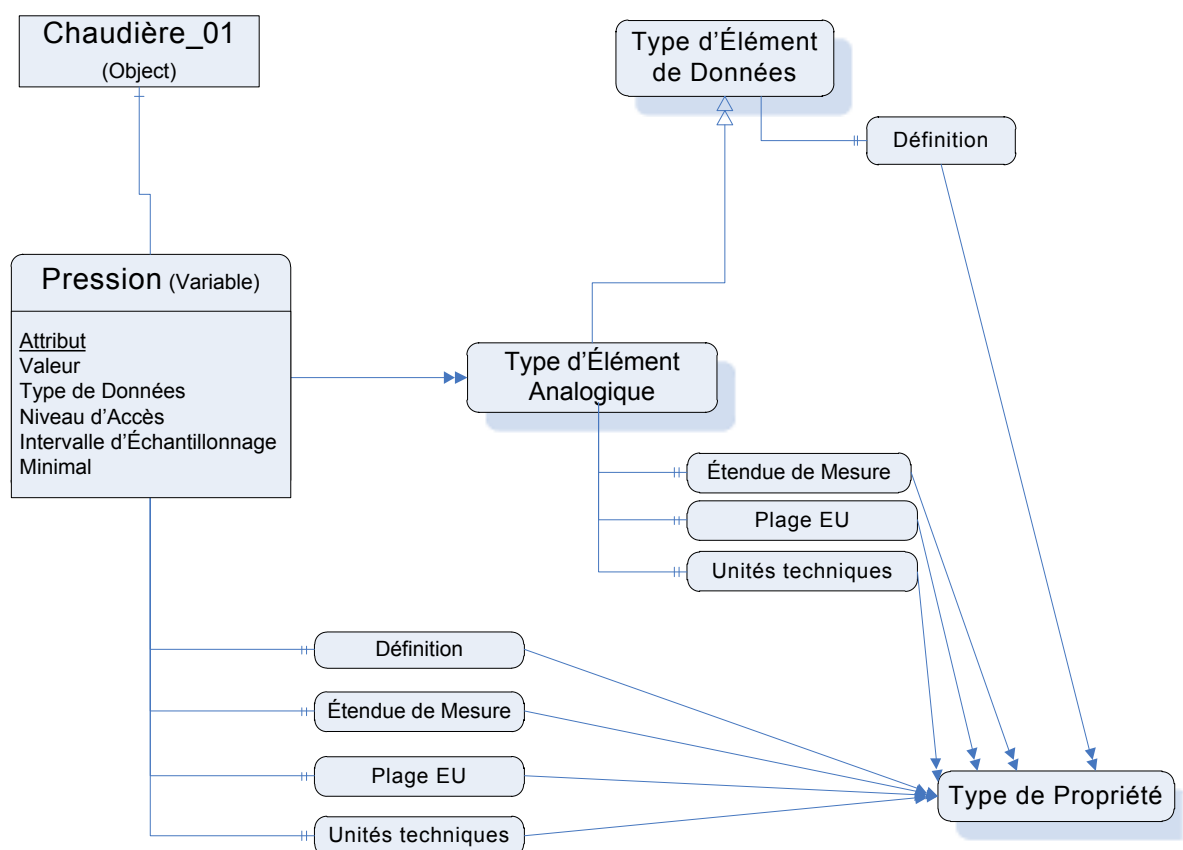
La *Propriété Définition de l'Axe* contient des informations sur les *Unités Techniques* et la *Plage* applicable à tous les axes.

Le bit de *Modifications Sémantiques du Code de Statut* doit être défini en cas de modification de l'une quelconque des *Propriétés Etendue de mesure*, *Plage EU*, *Unités Techniques*, *Titre*, *Définition de l'Axe* (voir 5.2 pour des informations supplémentaires).

5.4 Modèle d'Espace d'adresses

Les *Éléments de Données* sont toujours définis comme des composants de données d'autres *Nœuds* de l'*Espace d'adresses*. Ils ne sont jamais définis par eux-mêmes. Un exemple simple d'un conteneur pour les *Éléments de Données* serait un "Objet Dossier" mais il peut être un *Objet* d'un tout autre type.

La Figure 4 illustre le modèle d'*Espace d'adresses* de base d'un *Élément de Données*, dans ce cas un *Élément Analogique*.



IEC

Figure 4 – Représentation des Éléments de Données dans l'Espace d'adresses

Chaque *Élément de Données* est représenté par une *Variable de Données* avec un ensemble particulier d'*Attributs*. La référence *Définition de Type* indique le type de l'*Élément de Données* (dans ce cas, le *Type d'Élément Analogique*). Des caractéristiques supplémentaires des *Éléments de Données* sont définies en utilisant les *Propriétés*. Les *Types de Variables* donnés en 5.3 spécifient les propriétés qui peuvent exister. Ces *Propriétés* sont considérées comme utiles pour l'Accès aux Données pour une large plage de clients. Il convient que les serveurs qui souhaitent diffuser des informations similaires utilisent la *Propriété* OPC définie plutôt qu'une propriété spécifique au fournisseur.

La figure ci-dessus illustre uniquement un sous-ensemble d'*Attributs* et de *Propriétés*. D'autres *Attributs* définis pour les *Variables* dans l'UEC 62541-3. (par exemple, *Description*) peuvent également être disponibles.

5.5 Attributs des Éléments de Données

Ce paragraphe répertorie les *Attributs* des *Variables* qui revêtent une importance particulière pour l'Accès aux Données. Ils sont spécifiés en détail dans l'IEC 62541-3. Les *Attributs* suivants sont particulièrement importants pour l'Accès aux Données:

- Valeur (Value)
- Type de Données (DataType)
- Niveau d'Accès (AccessLevel)
- Intervalle d'Echantillonnage Minimal (MinimumSamplingInterval)

L'attribut *Valeur* est la valeur la plus récente de la *Variable* dont dispose le *Serveur*. Son type de données est défini par l'Attribut *Type de Données*. L'Attribut *Niveau d'Accès* définit

l'aptitude de base du *Serveur* à accéder aux données courantes et l'*Attribut Intervalle d'Échantillonnage Minimal* définit l'état courant des données.

Lorsqu'un client demande l'*Attribut Valeur* pour une opération de lecture ou de surveillance, le *Serveur* renvoie toujours un *Code de Statut* (la qualité et l'aptitude du *Serveur* à accéder et à fournir la valeur) et, de manière facultative, un *Horodatage Serveur* (*ServerTimestamp*) et/ou un *Horodatage Source* (*SourceTimestamp*), basés sur la demande du Client. Voir IEC 62541-4 pour de plus amples informations sur le *Code de Statut* et la signification des deux horodatages. Des codes de statut particuliers pour l'Accès aux Données sont définis en 6.3.

5.6 Types de Données

5.6.1 Aperçu général

Une description des *Types de Données* définis dans la présente spécification est détaillée ci-après.

Les *Types de Données* tels que *Chaîne*, *Booléen*, *Double* ou *Texte Localisé* sont définis dans l'IEC 62541-3. Leur représentation est spécifiée dans l'IEC 62541-5.

5.6.2 Plage

Cette structure définit la *Plage* applicable à une valeur. Ses éléments sont définis dans le Tableau 14.

Tableau 14 – Structure du DataType (Type de Données) «Plage»

Dénomination	Type	Description
Plage	structure	
bas	Double	Valeur la plus basse de la plage.
haut	Double	Valeur la plus haute de la plage.

Sa représentation dans l'*Espace d'adresses* est définie dans le Tableau 15.

Tableau 15 – Définition de Plage

Attributs	Valeur
BrowseName (Nom de navigation)	Plage

5.6.3 Information EU

Cette structure contient des informations sur les *Unités Techniques*. Ses éléments sont définis dans le Tableau 16.

Tableau 16 – Structure du DataType (Type de Données) Information EU

Dénomination	Type	Description
Information EU	structure	
Uri d'Espace de Nom	Chaîne	Identifie l'organisme (société, organisme de normalisation) qui définit l' <i>Information EU</i> .
identificateur d'Unité	Int32	Identificateur pour l'évaluation programmatique. –1 est utilisé si un <i>identificateur d'Unité</i> n'est pas disponible.
Nom d'Affichage	Texte Localisé	Le <i>Nom d'Affichage</i> de l'unité technique est généralement l'abréviation de l'unité technique, par exemple "h" pour heure ou "m/s" pour mètre par seconde.
description	Texte Localisé	Contient le nom complet de l'unité technique, par exemple "heure" ou "mètre par seconde".

Sa représentation dans l'*Espace d'adresses* est définie dans le Tableau 17.

Tableau 17 – Définition de l'Information EU

Attributs	Valeur
BrowseName	Information EU

Pour faciliter l'interopérabilité, l'OPC UA spécifie comment appliquer les "**Codes for Units of Measurement**" (**Codes pour les Unités de mesure**) largement acceptés, publiés par le "United Nations Centre for Trade Facilitation and Electronic Business" (organisme des Nations Unies pour la Facilitation des Procédures Commerciales et le Commerce Électronique) (voir UN/CEFACT: **Recommandation UNECE N°. 20**). L'OPC UA utilise et repose sur le système international d'unités (Unités SI), mais fournit en outre un code fixe qui peut être utilisé pour une évaluation automatisée. Cette recommandation a été acceptée par de nombreuses industries à un échelon mondial.

Le Tableau 18 contient un court extrait de l'Annexe publiée avec les listes de codes:

Tableau 18 – Exemples issus de la Recommandation UNECE

Extrait de la Recommandation N°. 20, Annexe 1			
Code commun	Dénomination	Facteur de conversion	Symbole
C81	radian		rad
C25	milliradian	10^{-3} rad	mrad
MMT	millimètre	10^{-3} m	mm
HMT	hectomètre	10^2 m	hm
KTM	kilomètre	10^3 m	km
KMQ	Kilogramme par mètre cube	kg/m^3	kg/m^3
FAH	degré Fahrenheit	$5/9 \times K$	°F
J23	degré Fahrenheit par heure	$1,543\,210 \times 10^{-4} \text{ K/s}$	°F/h

Les colonnes spécifiques de ce tableau doivent permettre de créer la structure de *Information EU* telle que définie par les règles suivantes:

- Le **Code commun** est représenté sous forme de longueur de variable alphanumérique de 3 caractères. Il doit être utilisé pour l'*Information EU.Id d'unité* (*EUInformation.unitId*). Le pseudocode suivant spécifie l'algorithme de conversion du Code commun en un code Int32 lorsque nécessaire pour l'*Information EU.Id d'unité*

```

Int32 unitId = 0;
Int32 c;
for (i=0; i<=3;i++)
{
    c = CommonCode[i];
    if (c == 0) break;           // fin du Code commun
    unitId = unitId << 8;
    unitId = unitId | c;
}

```

- Le champ **Symbole** doit être reproduit dans le champ *Information EU.Nom d'affichage* (*EUInformation.displayName*). Le champ Id de paramètre de lieu (locateld) de *Information EU.Nom d'affichage* doit être vide.

- Le champ **Dénomination** doit être utilisé pour *Information EU.description* (*EUInformation.description*). Lorsque la dénomination est reproduite, le champ Id de paramètre de lieu (locateld) de *Information EU.description* doit être vide. Si la dénomination est localisée, le champ Id de paramètre de lieu doit alors spécifier le paramètre de lieu correct.

Le champ *Information EU.Uri* de l'espace de nom (*EUInformation.namespaceUri*) doit être <http://www.opcfoundation.org/UA/units/un/cefact>.

NOTE L'utilisation de la Recommandation N° 20 telle que spécifiée présentera des avantages, car elle peut être interprétée du point de vue des programmes par les *Clients* OPC UA génériques. Cependant, la structure de *Information EU* a été définie de sorte que d'autres organismes de normalisation puissent intégrer leurs définitions relatives aux unités techniques à l'OPC UA. Lorsque les *Serveurs* utilisent ce type d'approche, ils doivent alors identifier ces organismes de normalisation au moyen d'un *Uri d'espace de nom* (*namespaceUri*) approprié dans *Information EU.Uri* de l'espace de nom.

5.6.4 Type de Nombre Complexe (ComplexNumberType)

Cette structure définit la valeur complexe à virgule flottante IEEE à 32 bits. Ses éléments sont définis dans le Tableau 19.

Tableau 19 – Structure du Type de Données Type de Nombre Complexe

Dénomination	Type	Description
Type de Nombre Complexe	structure	
réel	A virgule flottante	Partie réelle de la valeur
imaginaire	A virgule flottante	Partie imaginaire de la valeur

Sa représentation dans l'*Espace d'adresses* est définie dans le Tableau 20

Tableau 20 – Définition du Type de Nombre Complexe

Attributs	Valeur
BrowseName	Type de Nombre Complexe

5.6.5 Type de Nombre Complexe Double (DoubleComplexNumberType)

Cette structure définit la valeur complexe double IEEE à 64 bits. Ses éléments sont définis dans le Tableau 21.

Tableau 21 – Structure du Type de Données Type de Nombre Complexe Double

Dénomination	Type	Description
Type de Nombre Complexe Double	structure	
réel	Double	Partie réelle de la valeur
imaginaire	Double	Partie imaginaire de la valeur

Sa représentation dans l'*Espace d'adresses* est définie dans le Tableau 22.

Tableau 22 – Définition du Type de Nombre Complexe Double

Attributs	Value
BrowseName	Type de Nombre Complexe Double

5.6.6 Informations sur l'Axe (AxisInformation)

Cette structure définit les informations concernant l'axe auxiliaire pour les *Variables* de *Type d'Élément de Matrice*.

Il existe trois types habituels d'utilisation de cette structure:

- a) Le palier entre les points est constant et peut être prévu en utilisant les informations concernant la plage et le nombre de points. Dans ce cas, *Paliers d'axe* (*axisSteps*) peut être mis à NULL.
- b) Le palier entre les points n'est pas constant, mais reste identique pendant une longue période (d'une acquisition à une autre par exemple). Dans ce cas, *Paliers d'axe* contient la valeur de chaque palier sur l'axe.
- c) Le palier entre les points n'est pas constant et change à chaque mise à jour. Dans ce cas, un type tel que *Type de Matrice XY* doit être utilisé et *Paliers d'axe* est mis à NULL.

Ses éléments sont définis dans le Tableau 23.

Tableau 23 – Structure du Type de Données Informations sur l'Axe

Dénomination	Type	Description
Informations sur l'axe	structure	
Unités Techniques	Information EU	Contient les informations sur les unités techniques pour un axe donné.
Plage EU	Plage	Limites de la plage de l'axe
titre	Texte localisé	Intitulé de l'axe lisible par l'utilisateur, utile lorsque les unités sont données en %, l'intitulé peut être "Distribution granulométrique"
Type de Facteur d'Échelle selon un Axe	Énumération de Facteur d'Échelle selon un Axe	LINEAR, LOG, LN, définis par Paliers d'axe
Paliers d'axe	Double[]	Valeur spécifique de chaque palier d'axe, peut être mise à "Null" lorsqu'elle n'est pas utilisée

Lorsque les paliers de l'axe sont constants, *Paliers d'axe* peut être mis à "Null" et dans ce cas, les limites de *Plage* sont utilisées pour calculer les paliers. Le nombre de paliers sur l'axe est issu de l'*Élément de Matrice.Dimensions de Matrice* (*ArrayItem.ArrayDimensions*) parent.

5.6.7 Énumération de Facteur d'Échelle selon un Axe (AxisScaleEnumeration)

Cette énumération identifie le type d'axe sur lequel les données doivent être affichées. Ses valeurs sont définies dans le Tableau 24.

Tableau 24 – Valeurs de l'Énumération de Facteur d'Échelle selon un Axe

Valeur	Description
LINEAR_0	Échelle linéaire
LOG_1	Échelle logarithmique base 10
LN_2	Échelle logarithmique base e

Sa représentation dans l'*Espace d'adresses* est définie dans le Tableau 25.

Tableau 25 – Définition de l'Énumération de Facteur d'Échelle selon un Axe

Attributs	Valeur
BrowseName	Énumération de Facteur d'Échelle selon un Axe

5.6.8 Type XV (XVType)

Cette structure définit une valeur physique par rapport à un axe des X et est utilisée comme le *Type de Données* de la *Valeur* du *Type d'Élément de Matrice XY*. Pour des détails, voir 5.3.4.3.

De nombreux dispositifs peuvent produire des valeurs qui peuvent être parfaitement représentées par une valeur à virgule flottante IEEE à 32 bits, mais ils peuvent se positionner sur l'axe des X avec une précision qui exige une valeur double IEEE à 64 bits. Par exemple, la valeur de crête dans un spectre d'absorbance où l'amplitude de la crête peut être

représentée par une valeur à virgule flottante IEEE à 32 bits, mais la position de sa fréquence exige 10 chiffres, ce qui implique d'utiliser une valeur double IEEE à 64 bits.

Ses éléments sont définis dans le Tableau 26.

Tableau 26 – Structure du Type de Données Type XV

Dénomination	Type	Description
Type XV	structure	
x	Double	Position de cette valeur sur l'axe des X
valeur	À virgule flottante	Valeur elle-même

Sa représentation dans l'*Espace d'Adresses* est définie dans le Tableau 27.

Tableau 27 – Définition du Type XV

Attributs	Valeur
BrowseName	Type XV

6 Utilisation particulière des Services d'Accès aux Données

6.1 Généralités

L'IEC 62541-4 spécifie l'ensemble complet des services. Les services nécessaires pour l'Accès aux Données sont:

- Le Service *Vue* et le service *Consultation* destinés à détecter les *Éléments de Données* et leurs *Propriétés*.
- Le service *Attribut* destiné à lire ou écrire les *Attributs*, et notamment l'*Attribut Valeur*.
- Le service *Élément Surveillé* et le service *Abonnement* destinés à établir la surveillance des *Éléments de Données* et à recevoir des notifications de modification des données.

6.2 Pourcentage de Bande morte (PercentDeadBand)

Le *Filtre de Modification des Données (DataChangeFilter)* dans l'IEC 62541-4 définit les conditions dans lesquelles une notification de modification des données doit être indiquée. Ce filtre contient une *Valeur de bande morte (deadbandValue)* qui peut être de type *Bande morte Absolue (AbsoluteDeadband)* ou *Pourcentage de Bande morte (PercentDeadband)*. L'IEC 62541-4 spécifie déjà le comportement de la *Bande morte Absolue*. Le présent paragraphe spécifie le comportement du type *Pourcentage de Bande morte*.

Type de Bande morte (DeadBandType) = Pourcentage de Bande morte (PercentDeadBand)

Pour ce type de bande morte, la *Valeur de bande morte* est définie comme le pourcentage de la *Plage EU*. En d'autres termes, il ne s'applique qu'aux *Éléments Analogiques* avec une *Propriété Plage EU* qui définit la plage de valeur type pour l'élément. Cette plage doit être multipliée par la *Valeur de bande morte*, puis comparée à la modification de valeur réelle afin de déterminer la nécessité d'une notification de modification de données. Le pseudocode suivant montre comment la bande morte est calculée:

```
DataChange if (absolute value of (last cached value - current value) >
               (deadbandValue/100.0) * ((high-low) of EURange))
```

La plage de la *Valeur de bande morte* est comprise entre 0,0 et 100,0 en pourcentage. Une *Valeur de bande morte* spécifiée en dehors de cette plage sera refusée et indiquée avec le *Code de Statut* Bad_DeadbandFilterInvalid (voir le Tableau 28).

Si la Valeur de l'*Élément Surveillé* est une matrice, la logique de calcul de la bande morte doit alors être appliquée à chaque élément de la matrice. Si un élément qui exige une Modification de Donnée (DataChange) est déterminé, aucune vérification supplémentaire de la bande morte n'est nécessaire et toute la matrice doit être renvoyée.

6.3 Codes de statuts d'Accès aux Données

6.3.1 Aperçu général

Le présent paragraphe définit les codes et les règles supplémentaires qui s'appliquent au *Code de Statut* lorsqu'il est utilisé pour les valeurs d'Accès aux Données.

La structure générale du *Code de Statut* est spécifiée dans l'IEC 62541-4.. Elle comprend un ensemble de codes de résultats opérationnels communs qui s'appliquent également à l'Accès aux Données.

6.3.2 Codes de résultats de niveau opérationnel

Certaines conditions dans lesquelles une valeur de *Variable* a été générée ne sont valides que pour les données d'automatisation et notamment pour les données relatives aux appareils. Elles sont similaires, mais légèrement plus génériques que la description de la qualité des données dans les diverses spécifications de bus de terrain.

Dans les paragraphes suivants, le Tableau 28 contient les codes avec la sévérité MAUVAIS (BAD) qui indique une défaillance.

Le Tableau 29 contient les codes avec la sévérité DOUTEUX (UNCERTAIN) qui indique que la valeur a été générée dans des conditions anormales.

Le Tableau 30 contient les codes BON (GOOD) (réussite).

Noter ici encore qu'il s'agit des codes spécifiques à l'Accès aux Données et qu'ils complètent les codes qui s'appliquent à tous les types de données qui sont définis dans l'IEC 62541-4.

Tableau 28 – Codes de résultats de niveau opérationnel MAUVAIS

Identificateur symbolique	Description
Remarques: Le Code de Statut Mauvais (Bad) est défini dans l' IEC 62541-4. Il doit être utilisé lorsqu'il n'y a aucune raison particulière pour que la Valeur soit Mauvaise.	
Bad_ConfigurationError	Il existe un problème avec la configuration qui affecte l'utilité de la valeur.
Bad_NotConnected	Il convient que la variable reçoive sa valeur d'une autre variable, mais elle n'a jamais été configurée en conséquence.
Bad_DeviceFailure	Une défaillance s'est produite dans l'équipement/source de données qui a généré la valeur et qui a affecté la valeur.
Bad_SensorFailure	Une défaillance s'est produite dans le capteur duquel la valeur est dérivée par l'équipement/source de données. Les bits de limites sont utilisés pour définir si les limites de la valeur ont été atteintes.
Remarques: Bad_NoCommunication est défini dans l'IEC 62541-4. Il doit être utilisé lorsque les communications avec la source de données sont définies mais pas établies, et aucune dernière valeur connue n'est disponible.	
Bad_OutOfService	La source des données n'est pas opérationnelle.
Bad_LastKnown	L'OPC UA exige que le <i>Serveur</i> doit renvoyer une valeur Nulle lorsque la <i>Sévérité</i> est Mauvais. Par conséquent, le code de Bus de Terrain "Bad_LastKnown" doit être mis en correspondance avec Uncertain_NoCommunicationLastUsable
Bad_DeadbandFilterInvalid	Le <i>Pourcentage de Bande morte</i> spécifié n'est pas compris entre 0,0 et 100,0 ou un <i>Pourcentage de Bande morte</i> n'est pas pris en charge dans la mesure où une <i>Plage EU</i> n'est pas configurée.
Remarques:	

Identificateur symbolique	Description
•	Bad_WaitingForInitialData est défini dans l'IEC 62541-4.

Tableau 29 – Codes de résultats de niveau opérationnel DOUTEUX

Identificateur symbolique	Description
Remarques: Le Code de Statut Douteux est défini dans l'IEC 62541-4. Il doit être utilisé lorsqu'il n'y a aucune raison particulière pour que la Valeur soit Douteuse.	
Uncertain_NoCommunicationLastUsable	La communication avec la source de données a échoué. La valeur de la variable est la dernière valeur ayant présenté une bonne qualité, il n'est pas certain que cette valeur soit encore en vigueur. Dans ce cas, l'horodatage serveur est le dernier moment où l'état de la communication a été vérifié. Le moment auquel la valeur a été vérifiée en dernier comme vrai n'est plus disponible.
Uncertain_LastUsableValue	Quelle que soit la mise à jour, cette valeur a cessé de fonctionner. Ceci se produit lorsqu'une variable d'entrée est configurée pour recevoir sa valeur d'une autre variable et que cette configuration est effacée après réception d'une ou de plusieurs valeurs. Ce statut/sous-statut n'est pas utilisé pour indiquer qu'une valeur est périmée. Les données périmées peuvent être détectées par le client en consultant les horodatages.
Uncertain_SubstituteValue	La valeur est une valeur opérationnelle qui a été écrasée et remplacée manuellement.
Uncertain_InitialValue	La valeur est une valeur initiale d'une variable qui reçoit généralement sa valeur d'une autre variable. Ce statut/sous-statut n'est réglé que pendant la configuration lorsque la variable n'est pas opérationnelle (hors service).
Uncertain_SensorNotAccurate	La valeur se situe à l'une des limites du capteur. Les bits de limites définissent la limite qui a été atteinte. Également établi si l'appareil peut déterminer que le capteur présente une précision réduite (par exemple, analyseur dégradé), auquel cas les bits de limites indiquent que la valeur n'est pas limitée.
Uncertain_EngineeringUnitsExceeded	La valeur est en dehors de la plage des valeurs définie pour ce paramètre. Les bits de limites indiquent la limite qui a été atteinte ou dépassée.
Uncertain_SubNormal	La valeur est dérivée de plusieurs sources et dispose de moins de BONNES sources que le nombre exigé.

Tableau 30 – Codes de résultats de niveau opérationnel BON

Identificateur symbolique	Description
Remarques: Le Code de Statut Bon est défini dans l'IEC 62541-4. Il doit être utilisé en l'absence de conditions particulières.	
Good_LocalOverride	La valeur a été écrasée et remplacée. Ceci signifie en général que l'entrée a été déconnectée et qu'une valeur saisie manuellement a été «forcée».

6.3.3 Bits de limites (LimitBits)

Les 16 bits inférieurs du *Code de Statut* sont des fanions binaires qui contiennent des informations supplémentaires mais qui n'affectent pas la signification du *Code de Statut*. Pour les *Éléments de Données*, le champ *Bits de limites* revêt une importance toute particulière. Dans certains cas, comme par exemple une défaillance de capteur, il peut fournir des informations de diagnostic utiles.

Les serveurs qui ne prennent pas en charge la Limite sont tenus de mettre ce champ à 0.

INTERNATIONAL
ELECTROTECHNICAL
COMMISSION

3, rue de Varembé
PO Box 131
CH-1211 Geneva 20
Switzerland

Tel: + 41 22 919 02 11
Fax: + 41 22 919 03 00
info@iec.ch
www.iec.ch